

Machine Learning Notes*

Jiajie Cheng
Carnegie Mellon University
(Dated: April 30, 2026)

CONTENTS

I. Classification & Regression	2
A. Supervised Machine Learning	2
B. Algorithms for Classification	3
C. Model Selection & Experimental Design & Hyperparameter Optimization	8
II. Linear Models	9
A. Perceptron	9
B. Optimization Methods	11
C. Linear Regression	12
D. Logistic Regression	12
1. Convexity	14
2. Feature Engineering	15
3. Overfitting and Regularization	15
III. Neural Networks (NNs)	17
A. Objective Functions for Neural Networks	19
B. Backpropagation	21
C. Social Fairness	21
D. Probably Approximately Correct (PAC) Learning	23
E. VC Dimension	27
1. Why 4 Points in $\mathbb{R}^2$ Are Not Shattered by Linear Classifiers	27
F. Maximum Likelihood Estimation (MLE)	28
1. Loss function	28
G. Maximum A Posteriori Estimation (MAP)	29
H. Convolution Neural Networks (CNNs) and Recurrent Neural Networks (RNNs)	31
1. Representative LLM Scales	34
I. Transformer	35
IV. Reinforcement Learning	37
A. Reinforcement Learning	37
B. Policy Gradient	39
V. Learning Paradigms	44
A. Recommender Systems	44
B. Ensemble Methods: Boosting & Bagging	45
1. AdaBoost	45
C. Bagging	46
D. K-Means	48
E. Principal Component Analysis (PCA)	49

* Based on [CMU 10-601 Introduction to Machine Learning Lectures](#)

I. CLASSIFICATION & REGRESSION

A. Supervised Machine Learning

Medical Diagnosis Dataset

As a (supervised) binary classification task

	labels	features			
	allergic?	hives?	sneezing?	red eye?	has cat?
1	-	Y	N	N	N
2	-	N	Y	N	N
3	+	Y	Y	N	N
4	-	Y	N	Y	Y
5	+	N	Y	Y	N

Medical Diagnosis Dataset

As a (supervised) classification task

	labels	features			
	allergy	hives?	sneezing?	red eye?	has cat?
1	none	Y	N	N	N
2	none	N	Y	N	N
3	dust	Y	Y	N	N
4	none	Y	N	Y	Y
5	mold	N	Y	Y	N

FIG. 1: Example of a typical machine learning problem

1. Problem Setting

- Set of possible inputs, $\mathbf{x} \in \mathcal{X}$ (all possible patients)
- Set of possible outputs, $y \in \mathcal{Y}$ (all possible diagnoses)
- Exists an unknown target function, $c^* : \mathcal{X} \rightarrow \mathcal{Y}$ (the doctor's brain)
- Set, $\mathcal{H}$, of candidate hypothesis functions, $h : \mathcal{X} \rightarrow \mathcal{Y}$ (all possible decision trees)

2. **Learner is given** N training examples $\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$ where $y^{(i)} = c^*(\mathbf{x}^{(i)})$ (history of patients and their diagnoses)

3. **Learner produces** a hypothesis function, $\hat{y} = h(\mathbf{x})$, that best approximates unknown target function $y = c^*(\mathbf{x})$ on the training data $\mathcal{D}$ (a decision tree that best fits the history of patients and their diagnoses)

4. Learner is evaluated by

- Loss function $l : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$ measures how "bad" predictions $\hat{y} = h(\vec{x})$ are compared to $y = c^*(\vec{x})$
- Another dataset $\mathcal{D}_{\text{test}} = \{(\vec{x}^{(1)}, y^{(1)}), \dots, (\vec{x}^{(N')}, y^{(N')})\}$
- Evaluate the average loss of $h(\vec{x})$ on $\mathcal{D}_{\text{test}}$

• Loss function:

- Squared loss (for regression): $\ell(y, \hat{y}) = (y - \hat{y})^2$
- Binary or 0-1 loss (for classification): $\ell(y, \hat{y}) = \mathbb{1}(y \neq \hat{y}) = \begin{cases} 1, & \text{if } y \neq \hat{y} \\ 0, & \text{otherwise} \end{cases}$

- **Error rate:** the error rate of a hypothesis h on a dataset $\mathcal{D}$ is the average 0-1 loss:

$$\text{error}(h, \mathcal{D}) = \frac{1}{N} \sum_{n=1}^N \mathbb{1}(y^{(n)} \neq \hat{y}^{(n)}) \quad (\text{I.1})$$

the mean squared error is the average squared loss.

The most frequently used evaluation metric for classification is $\text{error}(h, \mathcal{D}_{\text{train}})$, $\text{error}(h, \mathcal{D}_{\text{test}})$, and $\text{error}_{\text{true}}(h)$.

B. Algorithms for Classification

Algorithm 1. **Majority Vote**: predict the most common label in the training dataset.

Algorithm 2. **Memorizer**: if a set of features exists in the training dataset, predict its corresponding label; otherwise, predict a random label.

Algorithm 3. **decision stump**: based on a single feature, x_d , predict the most common label in the training dataset among all data points that have the same value for x_d .

Algorithm 4. **decision tree**: recursively walk from root to a leaf, following the attribute values labeled on the branches, and return the label at the leaf.

- a splitting criterion is a function that measures the effectiveness of splitting on a particular attribute, our decision tree learner selects the attribute as the one that maximizes the splitting criterion. Common splitting criteria include: error rate (minimize); accuracy = 1 - error rate (maximize); Mutual information (maximize); Gini gain (maximize)

Algorithm 1 Majority Vote

```

1: procedure TRAIN( $\mathcal{D}$ )
2:    $V \leftarrow$  most common label in  $\mathcal{D}$ 
3: procedure PREDICT( $\mathcal{D}_{\text{test}}$ )
4:   for  $(\mathbf{x}^{(i)}, y^{(i)}) \in \mathcal{D}_{\text{test}}$  do
5:      $\hat{y}^{(i)} \leftarrow V$ 

```

Algorithm 2 Memorizer

```

1: procedure TRAIN( $\mathcal{D}$ )
2:   Store all training examples  $\mathcal{D}$ 
3: procedure PREDICT( $\mathbf{x}$ )
4:   if  $\mathbf{x}$  exists in  $\mathcal{D}$  then
5:      $\hat{y} \leftarrow$  corresponding label from  $\mathcal{D}$ 
6:   else
7:      $\hat{y} \leftarrow$  random label

```

Algorithm 3 Decision Stump

```

1: procedure TRAIN( $\mathcal{D}$ )
2:   Pick a feature dimension  $m$ 
3:   Divide dataset on  $x_m$ :

```

$$\mathcal{D}^{(0)} = \{(\mathbf{x}^{(i)}, y^{(i)}) \in \mathcal{D} : x_m^{(i)} = 0\}$$

$$\mathcal{D}^{(1)} = \{(\mathbf{x}^{(i)}, y^{(i)}) \in \mathcal{D} : x_m^{(i)} = 1\}$$

```

4:   Compute majority votes:

```

$$v^{(0)} = \text{majority_vote}(\mathcal{D}^{(0)})$$

$$v^{(1)} = \text{majority_vote}(\mathcal{D}^{(1)})$$

```

5: procedure PREDICT( $\mathbf{x}$ )
6:   if  $x_m = 0$  then
7:      $\hat{y} \leftarrow v^{(0)}$ 
8:   else
9:      $\hat{y} \leftarrow v^{(1)}$ 

```

Algorithm 4 Decision Tree

```

1: procedure TRAIN( $\mathcal{D}$ )
2:   root  $\leftarrow$  TRAINRECURSE( $\mathcal{D}$ )
3:   Store root
4: procedure TRAINRECURSE( $\mathcal{D}'$ )
5:   Create new node  $p$ 
6:   if (a) splitting criterion for all features is below threshold then
7:     or (b)  $\mathcal{D}'$  is empty
8:     or (c) all labels in  $\mathcal{D}'$  are identical
9:     or (d) for all features, all values in  $\mathcal{D}'$  are identical
10:     $p.type \leftarrow$  "leaf"
11:     $p.vote \leftarrow$  majority_vote( $\mathcal{D}'$ )
12:    return  $p$ 
13:  else
14:     $p.type \leftarrow$  "internal"
15:     $p.m \leftarrow \underset{m \in \{1, \dots, M\}}{\operatorname{argmax}} \text{ splitting\_criterion}(\mathcal{D}', m)$ 
16:    for each value  $v$  of feature  $p.m$  do
17:       $\mathcal{D}'_{x_m=v} \leftarrow \{(\mathbf{x}, y) \in \mathcal{D}' : x_m = v\}$ 
18:      child_v  $\leftarrow$  TRAINRECURSE( $\mathcal{D}'_{x_m=v}$ )
19:       $p.branches[v] \leftarrow$  child_v
20:  return  $p$ 

```

Recursive Training for Decision Trees

- def train(dataset $\mathcal{D}'$):
 - Let $p = \text{new Node}()$
 - **Base Case:** If (1) all labels $y^{(i)}$ in $\mathcal{D}'$ are identical (2) $\mathcal{D}'$ is empty (3) for each feature, all values are identical
 - $p.type = \text{Leaf}$ // The node p is a leaf node
 - $p.label = \text{majority_vote}(\mathcal{D}')$ // Store the label
 - return p
 - **Recursive Step:** Otherwise
 - Make an internal node
 - $p.type = \text{Internal}$ // The node p is an internal node
 - Pick the best feature X_m according to splitting criterion
 - $p.attr = \underset{m}{\operatorname{argmax}} \text{ splitting_criterion}(\mathcal{D}', X_m)$ // Store the feature on which to split
 - For each value v of feature X_m :
 - $\mathcal{D}_{X_m=v} = \{(\mathbf{x}, y) \in \mathcal{D}' : x_m = v\}$ // Select a partition of the data
 - child_v = train($\mathcal{D}_{X_m=v}$) // Recursively build the child
 - $p.branches[v] = \text{child}_v$ // Create a branch with label v
 - return p

41

The **entropy** of a random variable describes the uncertainty of its outcome: the higher the entropy, the less certain we are about what the outcome will be.

$$H(X) = - \sum_{v \in V(X)} P(X = v) \log_2(P(X = v)) \quad (\text{I.2})$$

where X is a (discrete) random variable, $V(X)$ is the set of possible values X can take on.

The entropy of a set describes how uniform or pure it is: the higher the entropy, the more impure or "mixed-up" the set is

$$H(S) = - \sum_{v \in V(S)} \frac{|S_v|}{|S|} \log_2 \left(\frac{|S_v|}{|S|} \right) \quad (\text{I.3})$$

where S is a collection of values, $V(S)$ is the set of unique values in S , S_v is the collection of elements in S with value v . If S is split fifty-fifty between two values, then $H(S) = -\frac{1}{2} \log_2 \left(\frac{1}{2}\right) - \frac{1}{2} \log_2 \left(\frac{1}{2}\right) = 1$ bit. If S is pure, then $H(S) = -1 \cdot \log_2(1) = 0$ bit.

The **mutual information** between a feature and the label describes how much clarity knowing the feature provides about the label

$$\begin{aligned} I(y; x_d) &= H(y) - H(y | x_d) \\ &= H(y) - \sum_{v \in V(x_d)} f_v * H(Y_{x_d=v}) \end{aligned} \quad (I.4)$$

where x_d is a feature and y is the set of all labels $V(x_d)$ is the set of possible values x_d can take on, f_v is the fraction of data points where $x_d = v$, and $Y_{x_d=v}$ is the set of all labels where $x_d = v$

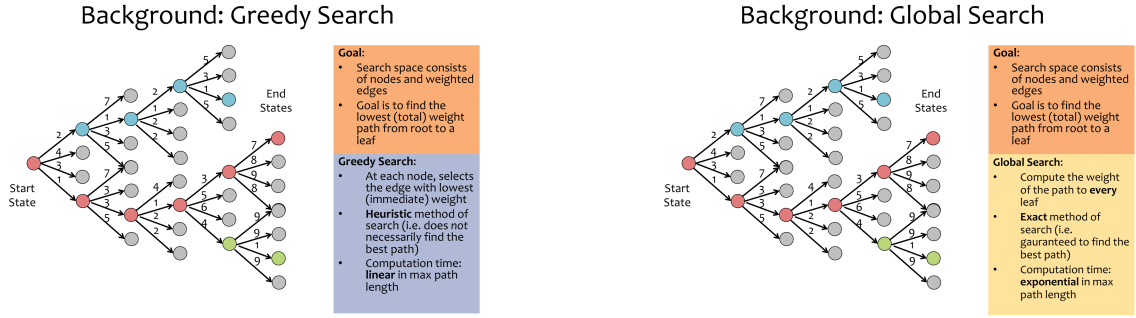


FIG. 2: Greedy Search (heuristic) vs. Global Search (exact)

- search space: all possible decision trees
- node: single decision tree
- edge: connects one full tree to another, where child has one more split than parent
- edge weight: (negative) splitting criterion
- DT learning: greedy search, maximizing our splitting criterion at each step where child has one more split than parent

$$\text{Amount of overfitting} = \text{error}_{\text{true}}(h) - \text{error}(h, D_{\text{train}}) \quad (I.5)$$

Avoid overfitting:

Step 1. Do not grow tree beyond some maximum depth

Step 2. Do not split if splitting criterion (e.g. mutual information) is below some threshold

Step 3. Stop growing when the split is not statistically significant

Step 4. Grow the entire tree, then prune (e.g. Reduced Error Pruning)

- Split data in two: training dataset and validation dataset; Grow the full tree using the training dataset
- Repeatedly prune the tree: Evaluate each split using a validation dataset by comparing the validation error rate with and without that split; (Greedy) remove the split that most decreases the validation error rate; Stop if no split improves validation error, otherwise repeat

Overfitting and Underfitting

Underfitting

- The model...
 - is too simple
 - is unable captures the trends in the data
 - exhibits too much bias
- *Example:* majority-vote classifier (i.e. depth-zero decision tree)
- *Example:* a toddler (that has **not** attended medical school) attempting to carry out medical diagnosis

Overfitting

- The model...
 - is too complex
 - is fitting the noise in the data or fitting “outliers”
 - does not have enough bias
- *Example:* our “memorizer” algorithm responding to an irrelevant attribute
- *Example:* medical student who simply memorizes patient case studies, but does not understand how to apply knowledge to new patients

29

FIG. 3: Overfitting vs. Underfitting

Overfitting in Decision Tree Learning

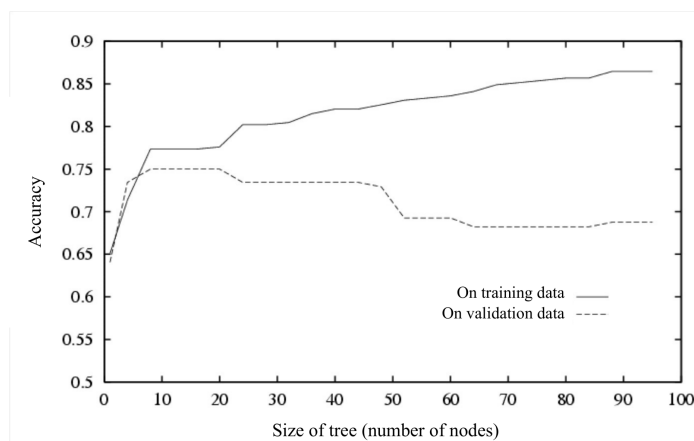


Figure from Tom Mitchell

33

FIG. 4: Overfitting happens when a model is too complex (more splits)

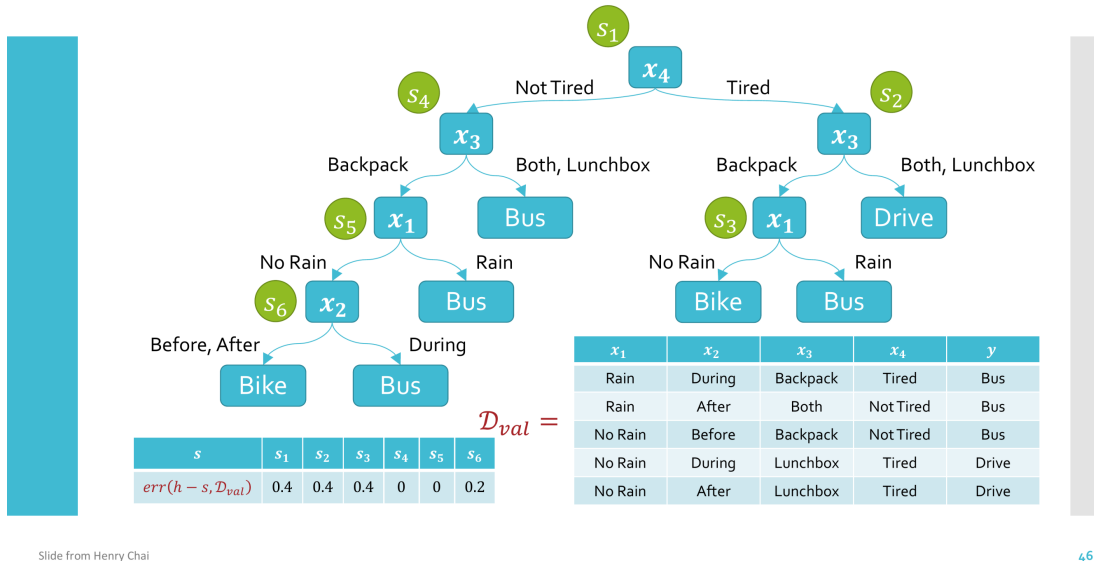


FIG. 5: Pruning: compare the error rates after a certain node is removed.

Decision Trees (DTs) in the Wild

- DTs are one of the most popular classification methods for practical applications
 - Reason #1: The learned representation is **easy to explain** a non-ML person
 - Reason #2: They are **efficient** in both computation and memory
- DTs can be applied to a wide variety of problems including **classification**, **regression**, **density estimation**, etc.
- **Applications of DTs** include...
 - medicine, molecular biology, text classification, manufacturing, astronomy, agriculture, and many others
- **Decision Forests** learn many DTs from random subsets of features; the result is a very powerful example of an **ensemble method** (discussed later in the course)

Decision Trees: Pros & Cons

Pros

- Interpretable
- Efficient (computational cost and storage)
- Can be used for classification and regression tasks
- Compatible with categorical and real-valued features

Cons

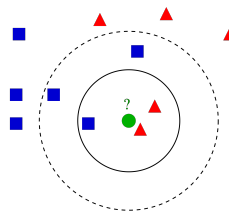
- Greedy: each split only considers the immediate impact
- Not guaranteed to find the smallest (fewest number of splits) tree that achieves a training error rate of 0.
- Liable to overfit!

Algorithm 5 k-Nearest Neighbors

- ```

1: procedure TRAIN($\mathcal{D}$)
2: Store all training examples $\mathcal{D}$
3: procedure PREDICT($\mathbf{x}_{\text{test}}$)
4: Compute distances: $d_i = \|\mathbf{x}_{\text{test}} - \mathbf{x}^{(i)}\|$ for all
 i
5: Find $\mathcal{N}_k =$ indices of k smallest distances
6: $\hat{y}_{\text{test}} \leftarrow \underset{c}{\operatorname{argmax}} \sum_{i \in \mathcal{N}_k} \mathbb{1}(y^{(i)} = c)$
7: return $\hat{y}_{\text{test}}$

```



## Theorem I.1: KNN: Theoretical Guarantees

Cover & Hart (1967)

Let  $h(\mathbf{x})$  be a Nearest Neighbor ( $k = 1$ ) binary classifier. As the number of training examples  $N$  goes to infinity:

$$\text{error}_{\text{true}}(h) < 2 \times \text{Bayes Error Rate} \quad (\text{I.6})$$

"In this sense, it may be said that half the very informally, Bayes Error Rate can be thought of as: 'the

best you could possibly do' classification information in an infinite sample set is contained in the nearest neighbor."

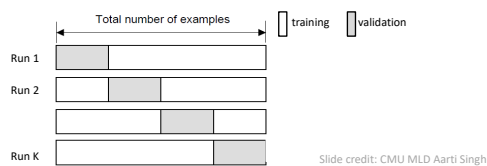
### C. Model Selection & Experimental Design & Hyperparameter Optimization

Use many "validation" datasets to select the best (external) hyperparameters. (N-fold cross validation)

#### Cross-validation

##### K-fold cross-validation

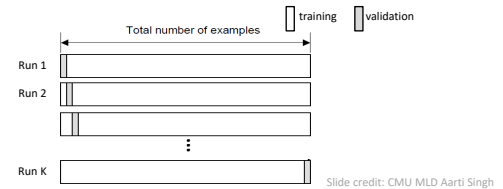
Create K-fold partition of the dataset.  
Do K runs: train using K-1 partitions and calculate validation error on remaining partition (rotating validation partition on each run).  
Report average validation error



#### Cross-validation

##### Leave-one-out (LOO) cross-validation (K=N-fold cross-validation)

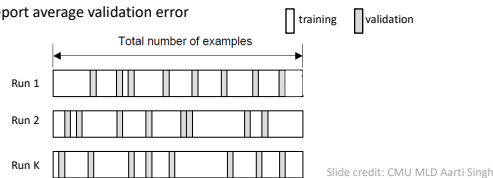
Special case of K-fold with K=N partitions  
Equivalently, train on N-1 samples and validate on only one sample per run for N runs



#### Cross-validation

##### Random subsampling

Randomly subsample a fixed fraction  $\alpha N$  ( $0 < \alpha < 1$ ) of the dataset for validation.  
Compute validation error with remaining data as training data.  
Repeat K times  
Report average validation error



## Experimental Design

|                             | Input                                                                                                            | Output                                                                   | Notes                                                                                                                        |
|-----------------------------|------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Training                    | <ul style="list-style-type: none"> <li>training dataset</li> <li>hyperparameters</li> </ul>                      | <ul style="list-style-type: none"> <li>best model parameters</li> </ul>  | We pick the best model parameters by learning on the training dataset for a fixed set of hyperparameters                     |
| Hyperparameter Optimization | <ul style="list-style-type: none"> <li>training dataset</li> <li>validation dataset</li> </ul>                   | <ul style="list-style-type: none"> <li>best hyperparameters</li> </ul>   | We pick the best hyperparameters by learning on the training data and evaluating error on the validation error               |
| Cross-Validation            | <ul style="list-style-type: none"> <li>training dataset</li> <li>validation dataset</li> </ul>                   | <ul style="list-style-type: none"> <li>cross-validation error</li> </ul> | We estimate the error on held out data by repeatedly training on N-1 folds and predicting on the held-out fold               |
| Testing                     | <ul style="list-style-type: none"> <li>test dataset</li> <li>hypothesis (i.e. fixed model parameters)</li> </ul> | <ul style="list-style-type: none"> <li>test error</li> </ul>             | We evaluate a hypothesis corresponding to a decision rule with fixed model parameters on a test dataset to obtain test error |

## II. LINEAR MODELS

### A. Perceptron

From linear algebra, we could use the convention of column vectors,

$$\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_D \end{bmatrix} \text{ and } \mathbf{a}^T = [a_1 \ a_2 \ \cdots \ a_D] \quad (\text{II.1})$$

The dot product between two  $D$ -dimensional vectors is

$$\vec{\mathbf{a}} \cdot \vec{\mathbf{b}} = \mathbf{a}^T \mathbf{b} = [a_1 \ a_2 \ \cdots \ a_D] \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_D \end{bmatrix} = \sum_{d=1}^D a_d b_d \quad (\text{II.2})$$

The  $L2$ -norm of  $\mathbf{a} = \|\mathbf{a}\|_2 = \sqrt{\mathbf{a}^T \mathbf{a}}$  Two vectors are orthogonal if

$$\mathbf{a}^T \mathbf{b} = 0 \quad (\text{II.3})$$

Linear Functions  $y = \mathbf{w}^T \mathbf{x} + b$  will have such linear decision boundaries,

$$y = \text{sign}(\mathbf{w}^T \mathbf{x} + b) \quad (\text{II.4})$$

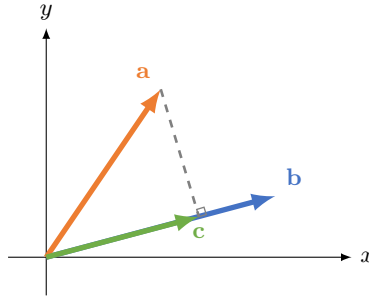


FIG. 6: Vector Projection:  $\mathbf{c}$  is the projection of  $\mathbf{a}$  onto  $\mathbf{b}$

The projection of  $\mathbf{a}$  onto  $\mathbf{b}$  is given by  $\mathbf{c}$ , and there are some equivalent expressions for  $\mathbf{c}$ :

$$\mathbf{c} = (\mathbf{a}^T \mathbf{b}) \mathbf{b} = \frac{\mathbf{a}^T \mathbf{b}}{\mathbf{b}^T \mathbf{b}} \mathbf{b} = \frac{\mathbf{a}^T \mathbf{b}}{\|\mathbf{b}\|_2^2} \mathbf{b} \quad (\text{II.5})$$

| Dimension | Linear Decision Boundary |
|-----------|--------------------------|
| 1D        | Point                    |
| 2D        | Line                     |
| 3D        | Plane                    |
| 4+D       | Hyperplane               |

A hyperplane creates two halfspaces:

$$\begin{aligned} \mathcal{S}_+ &= \{\mathbf{x} : \mathbf{w}^T \mathbf{x} + b > 0\} \\ \mathcal{S}_- &= \{\mathbf{x} : \mathbf{w}^T \mathbf{x} + b < 0\} \end{aligned} \quad (\text{II.6})$$

**Online learning:** different from batch learning, where the model is trained on the entire dataset at once, online learning updates the model incrementally as new data arrives. This is particularly useful when dealing with large datasets or streaming data, as it allows the model to adapt to new information without needing to retrain from scratch.

---

**Algorithm 6** Online Perceptron Algorithm
 

---

```

1: Initialize $\omega \leftarrow [0, 0, \dots, 0]^\top \in \mathbb{R}^M$
2: $b \leftarrow 0$ ▷ bias term
3: for $i = 1, 2, 3, \dots$ do
4: Receive unlabeled instance $\mathbf{x}^{(i)} \in \mathbb{R}^M$ ▷ for batch learning, we can shuffle the dataset and loop it
5: Predict $\hat{y} \leftarrow \text{sign}(\omega^\top \mathbf{x}^{(i)} + b)$
6: Receive true label $y^{(i)} \in \{+1, -1\}$
7: if $\hat{y} \neq y^{(i)}$ then ▷ if prediction is wrong
8: $\omega \leftarrow \omega + y^{(i)} \mathbf{x}^{(i)}$
9: $b \leftarrow b + y^{(i)}$
10: else
11: continue ▷ don't update

```

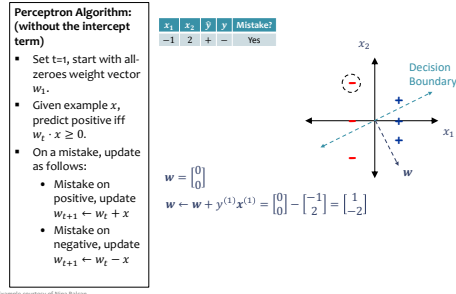
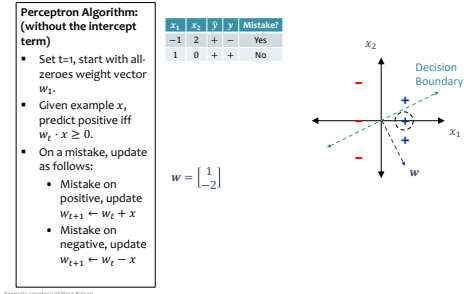
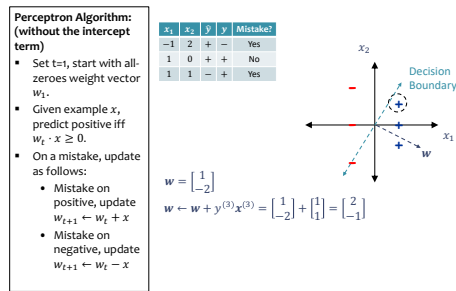
---

where  $\text{sign}(c) = \begin{cases} +1 & \text{if } c \geq 0 \\ -1 & \text{otherwise} \end{cases}$  is the sign function that maps the output of the linear function to a binary class label.

More precisely, the update rule can be written as:

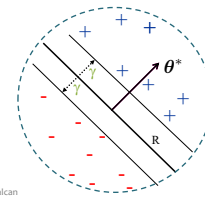
$$\text{If } \hat{y} \neq y^{(i)} \text{ and } y^{(i)} = +1: \quad \omega \leftarrow \omega + \mathbf{x}^{(i)}, \quad b \leftarrow b + 1 \quad (\text{II.7a})$$

$$\text{If } \hat{y} \neq y^{(i)} \text{ and } y^{(i)} = -1: \quad \omega \leftarrow \omega - \mathbf{x}^{(i)}, \quad b \leftarrow b - 1 \quad (\text{II.7b})$$

**(Online) Perceptron Algorithm: Example**

Example courtesy of Nina Balcan
**(Online) Perceptron Algorithm: Example**

43
Example courtesy of Nina Balcan
44
**(Online) Perceptron Algorithm: Example**

Example courtesy of Nina Balcan
46
**Perceptron Mistake Bound**

**Guarantee:** if some data has margin  $\gamma$  and all points lie inside a ball of radius  $R$  rooted at the origin, then the online Perceptron algorithm makes  $\leq (R/\gamma)^2$  mistakes

(Normalized margin: multiplying all points by 100, or dividing all points by 100, doesn't change the number of mistakes! The algorithm is invariant to scaling.)



**Main Takeaway:** For linearly separable data, if the perceptron algorithm cycles repeatedly through the data, it will **converge** in a finite # of steps.

76
Slide adapted from Nina Balcan

FIG. 7: Schematics of the Online Perceptron Algorithm

## B. Optimization Methods

Method 1. Close-form solution;

Method 2. Grid search;

Method 3. Random search (guessing);

Method 4. Gradient-based optimization (e.g., gradient descent, stochastic gradient descent, mini-batch SGD).

---

### Algorithm 7 Gradient Descent

---

```

1: procedure GD($\mathcal{D}, \theta^{(0)}$)
2: $\theta \leftarrow \theta^{(0)}$
3: while not converged do
4: $\theta \leftarrow \theta - \gamma \nabla J(\theta)$
5: return θ

```

---



---

### Algorithm 8 Stochastic Gradient Descent

---

```

1: procedure SGD($\mathcal{D}, \theta^{(0)}$)
2: $\theta \leftarrow \theta^{(0)}$
3: while not converged do
4: for $i \sim \text{shuffle}(\{1, 2, \dots, N\})$ do
5: $\theta \leftarrow \theta - \gamma \nabla_{\theta} J^{(i)}(\theta)$
6: return θ

```

---

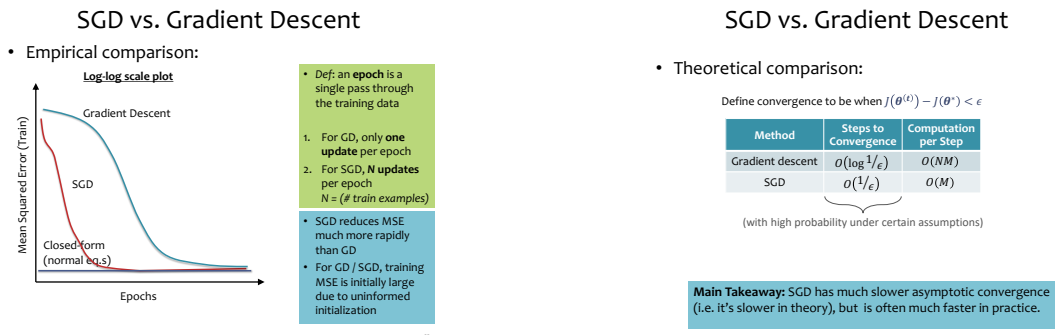


FIG. 8: SGD comparison with GD.

$\epsilon$  is the convergence threshold or tolerance;  $\gamma$  is the learning rate;  $N$ : number of training examples;  $M$ : number of parameters in  $\theta$ , which is equivalent to the number of features in simple models. SGD updates are high in variance and may not go in the direction of the true gradient, but they can help escape local minima and saddle points and can be much faster than GD for large datasets.

### C. Linear Regression

From linear regression objective function,

$$\begin{aligned}
 J(w, b) &= \frac{1}{N} \sum_i^N \left( y^{(i)} - \left( b + \sum_j^M w_j x_j^{(i)} \right) \right)^2 \\
 &= \frac{1}{N} \sum_i^N \left( y^{(i)} - \left( \boldsymbol{\theta}^\top \mathbf{x}^{(i)} \right) \right)^2 \\
 &= \frac{1}{N} \|\mathbf{y} - X\boldsymbol{\theta}\|_2^2
 \end{aligned} \tag{II.8}$$

Expansion of the objective function:

$$\begin{aligned}
 J(\boldsymbol{\theta}) &= \frac{1}{N} \|\mathbf{y} - X\boldsymbol{\theta}\|_2^2 \\
 &= \frac{1}{N} (\mathbf{y} - X\boldsymbol{\theta})^\top (\mathbf{y} - X\boldsymbol{\theta}) \\
 &= \frac{1}{N} (\mathbf{y}^\top - \boldsymbol{\theta}^\top X^\top) (\mathbf{y} - X\boldsymbol{\theta}) \\
 &= \frac{1}{N} (\mathbf{y}^\top \mathbf{y} - \boldsymbol{\theta}^\top X^\top \mathbf{y} - \mathbf{y}^\top X \boldsymbol{\theta} + \boldsymbol{\theta}^\top X^\top X \boldsymbol{\theta}) \\
 &= \frac{1}{N} (\mathbf{y}^\top \mathbf{y} - 2\boldsymbol{\theta}^\top X^\top \mathbf{y} + \boldsymbol{\theta}^\top X^\top X \boldsymbol{\theta})
 \end{aligned} \tag{II.9}$$

The derivative of the objective function is:

$$\begin{aligned}
 \nabla J(\boldsymbol{\theta}) &= \frac{1}{N} (0 - 2X^\top \mathbf{y} + 2\boldsymbol{\theta}^\top X^\top X) \\
 &= \frac{1}{N} (0 - 2X^\top \mathbf{y} + 2X^\top X \boldsymbol{\theta}) \\
 &= \frac{2}{N} (-X^\top \mathbf{y} + X^\top X \boldsymbol{\theta})
 \end{aligned} \tag{II.10}$$

Therefore, the close-form solution is taking the derivative to be zero, and rearranging the equation we find the parameter  $\boldsymbol{\theta}$  that minimizes the objective function:

$$\boldsymbol{\theta} = (X^\top X)^{-1} X^\top \mathbf{y} \tag{II.11}$$

### D. Logistic Regression

To avoid the problem of undifferentiable 0-1 loss, we can use the logistic loss function:

Sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{II.12}$$

and its derivative is:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \tag{II.13}$$

Let:

$$z^{(i)} = \boldsymbol{\theta}^\top \mathbf{x}^{(i)}, \quad \hat{y}^{(i)} = \sigma(z^{(i)}) \tag{II.14}$$

|                                                                  | Numerator layout         | Denominator layout     | Notes                         |
|------------------------------------------------------------------|--------------------------|------------------------|-------------------------------|
| $\frac{\partial}{\partial \mathbf{v}} \mathbf{v}$                | $I_N$                    | $I_N$                  | $\mathbf{v} \in \mathbb{R}^N$ |
| $\frac{\partial}{\partial \mathbf{v}} \mathbf{v}^T$              | $I_N$                    | $I_N$                  | $\mathbf{v} \in \mathbb{R}^N$ |
| $\frac{\partial}{\partial \mathbf{v}} t\mathbf{v}$               | $tI_N$                   | $tI_N$                 | $\mathbf{v} \in \mathbb{R}^N$ |
| $\frac{\partial}{\partial \mathbf{u}} \mathbf{u}^T \mathbf{v}$   | $\mathbf{v}^T$           | $\mathbf{v}$           |                               |
| $\frac{\partial}{\partial \mathbf{v}} \mathbf{u}^T \mathbf{v}$   | $\mathbf{u}^T$           | $\mathbf{u}$           |                               |
| $\frac{\partial}{\partial \mathbf{v}} \mathbf{v}^T \mathbf{v}$   | $2\mathbf{v}^T$          | $2\mathbf{v}$          |                               |
| $\frac{\partial}{\partial \mathbf{v}} \mathbf{v}^T A \mathbf{v}$ | $\mathbf{v}^T (A + A^T)$ | $(A + A^T) \mathbf{v}$ |                               |
| $\frac{\partial}{\partial \mathbf{v}} \mathbf{v}^T A \mathbf{v}$ | $2\mathbf{v}^T A$        | $2A \mathbf{v}$        | If $A = A^T$                  |
| $\frac{\partial}{\partial \mathbf{v}} A \mathbf{v}$              | $A$                      | $A^T$                  |                               |
| $\frac{\partial}{\partial \mathbf{v}} \mathbf{v}^T A$            | $A^T$                    | $A$                    |                               |

TABLE I: Denominator layout: gradient is a column vector (Commonly used in ML); Numerator layout: gradient is a row vector (Transpose of Denominator layout)

## Logistic Regression

|                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Data:</b> Inputs are continuous vectors of length M. Outputs are discrete.<br>$\mathcal{D} = \{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^N$ where $\mathbf{x} \in \mathbb{R}^M$ and $y \in \{0, 1\}$ |
| <b>Model:</b> Logistic function applied to dot product of parameters with input vector.<br>$p_{\theta}(y = 1 \mathbf{x}) = \frac{1}{1 + \exp(-\boldsymbol{\theta}^T \mathbf{x})}$               |
| <b>Learning:</b> finds the parameters that minimize some objective function. $\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} J(\boldsymbol{\theta})$             |
| <b>Prediction:</b> Output is the most probable class.<br>$\hat{y} = \underset{y \in \{0,1\}}{\operatorname{argmax}} p_{\theta}(y \mathbf{x})$                                                   |

53

FIG. 9: Logistic Regression

The log-likelihood function of a Bernoulli distribution is :

$$\begin{aligned}
 \ell(\boldsymbol{\theta}) &= \sum_{i=1}^N \log P\left(y^{(i)} \mid \mathbf{x}^{(i)}; \boldsymbol{\theta}\right) \\
 &= \sum_{i=1}^N \log \left( \sigma\left(\boldsymbol{\theta}^T \mathbf{x}^{(i)}\right) \right)^{y^{(i)}} \left( 1 - \sigma\left(\boldsymbol{\theta}^T \mathbf{x}^{(i)}\right) \right)^{1-y^{(i)}} \\
 &= \sum_{i=1}^N y^{(i)} \log \sigma\left(\boldsymbol{\theta}^T \mathbf{x}^{(i)}\right) + \left(1 - y^{(i)}\right) \log \left(1 - \sigma\left(\boldsymbol{\theta}^T \mathbf{x}^{(i)}\right)\right)
 \end{aligned} \tag{II.15}$$

Therefore, the objective function of logistic regression is the negative log-likelihood function:

$$J(\boldsymbol{\theta}) = -\frac{1}{N} \ell(\boldsymbol{\theta}) \tag{II.16}$$

and we may find the optimal parameter  $\theta$  by gradient descent or stochastic gradient descent.

$$\begin{aligned}
\nabla_{\theta} J(\theta) &= -\frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \ell^{(i)} \\
&= -\frac{1}{N} \sum_{i=1}^N \frac{\partial \ell^{(i)}}{\partial \hat{y}^{(i)}} \cdot \frac{\partial \hat{y}^{(i)}}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial \theta} \\
&= -\frac{1}{N} \sum_{i=1}^N \left( \frac{y^{(i)}}{\hat{y}^{(i)}} - \frac{1 - y^{(i)}}{1 - \hat{y}^{(i)}} \right) \hat{y}^{(i)} (1 - \hat{y}^{(i)}) \mathbf{x}^{(i)} \\
&= -\frac{1}{N} \sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)}) \mathbf{x}^{(i)} \\
&= -\frac{1}{N} \sum_{i=1}^N (y^{(i)} - \sigma(\theta^T \mathbf{x}^{(i)})) \mathbf{x}^{(i)}
\end{aligned} \tag{II.17}$$

There is no closed-form solution for logistic regression, so we need to use gradient-based optimization methods to find the optimal parameters. The stochastic gradient descent algorithm for logistic regression is shown in Algorithm 9.

---

**Algorithm 9** SGD for Logistic Regression

---

```

1: procedure SGD($\mathcal{D}, \theta^{(0)}$)
2: $\theta \leftarrow \theta^{(0)}$ ▷ Initialize parameters
3: while not converged do
4: for $i \sim \text{shuffle}(\{1, 2, \dots, N\})$ do do
5: $\mathbf{g} \leftarrow (\sigma(\theta^T \mathbf{x}^{(i)}) - y^{(i)}) \mathbf{x}^{(i)}$ ▷ Compute the gradient of the loss for the i -th example
6: where $\sigma(c) := 1/(1 + \exp(-c))$
7: $\theta \leftarrow \theta - \gamma \mathbf{g}$ ▷ Update parameters using the gradient
8: return θ

```

---

### 1. Convexity

Function  $f : \mathbb{R}^M \rightarrow \mathbb{R}$  is convex if  $\forall \mathbf{x}_1 \in \mathbb{R}^M, \mathbf{x}_2 \in \mathbb{R}^M, 0 \leq t \leq 1$  :

$$f(t\mathbf{x}_1 + (1-t)\mathbf{x}_2) \leq tf(\mathbf{x}_1) + (1-t)f(\mathbf{x}_2) \tag{II.18}$$

This is the famous Jensen's inequality! If  $f$  is convex, then the line segment between any two points on the graph of  $f$  lies above the graph (also including overlapping). If  $f$  is strictly convex, then the line segment lies strictly above the graph, except at the endpoints (which means the equation holds with strict inequality).

| Convex Function                        | Non-convex Function                                         |
|----------------------------------------|-------------------------------------------------------------|
| Each local minimum is a global minimum | May have multiple local minima, which are not global minima |

TABLE II: Convex vs. Non-convex Functions

The objective function for Logistic Regression, which we minimize for learning the parameters, is convex!

## 2. Feature Engineering

- One-hot encoding for categorical features: convert each category into a binary vector, where only one element is 1 and the rest are 0.
- Tokenization for text data: Tokens are pieces of text created by algorithms like byte pair encoding; they can represent whole words, parts of words, characters, or punctuation.
- Feature engineering: Humans decide what the features may be useful and implement the feature extraction process; Feature learning: Humans choose data and performance measure and allow the machine to automatically discover the representations needed for feature detection or classification from raw data.

## 3. Overfitting and Regularization

The problem of overfitting is when the model captures the **noise** in the training data instead of the **underlying structure**. On the opposite, underfitting is when the model is too simple to capture the underlying structure. See Figure 3.

Based on Occam's Razor: prefer the simplest hypothesis. For a simple case: it could be the one with fewer features, or "important" features. Regularization is a technique to prevent overfitting by adding a penalty term to the loss function, which encourages the model to be simpler.

We can define regularizer  $r(\theta)$  that we will add to our minimization objective to keep the model simple. For small  $r(\theta)$ , the model is simple; for large  $r(\theta)$ , the model is complex. The goal is find

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} J(\theta) + \lambda r(\theta)$$

where  $\lambda$  is the regularization strength, which controls the trade-off between fitting the training data and keeping the model simple. When  $\lambda$  is large, the system prefers simpler models, because the argmin will be dominated by the regularization term; when  $\lambda$  is small, the system prefers models that fit the training data well, because the argmin will be dominated by the original loss function  $J(\theta)$ .

- Add an L2 regularizer to Linear Regression (aka. Ridge Regression)

$$\begin{aligned} J_{\text{RR}}(\theta) &= J(\theta) + \lambda \|\theta\|_2^2 \\ &= \frac{1}{N} \sum_{i=1}^N \frac{1}{2} \left( \theta^T \mathbf{x}^{(i)} - y^{(i)} \right)^2 + \lambda \sum_{m=1}^M \theta_m^2 \end{aligned} \quad (\text{II.19})$$

- Add an L1 regularizer to Linear Regression (aka. LASSO Regression)

$$\begin{aligned} J_{\text{LASSO}}(\theta) &= J(\theta) + \lambda \|\theta\|_1 \\ &= \frac{1}{N} \sum_{i=1}^N \frac{1}{2} \left( \theta^T \mathbf{x}^{(i)} - y^{(i)} \right)^2 + \lambda \sum_{m=1}^M |\theta_m| \end{aligned} \quad (\text{II.20})$$

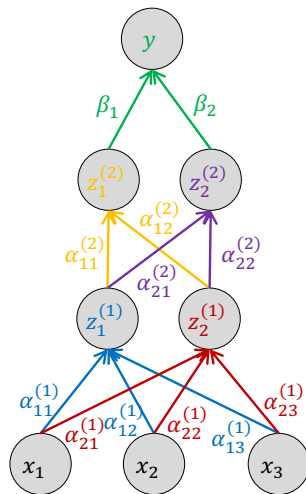
- Add an L2 regularizer to Logistic Regression

$$\begin{aligned} J'(\theta) &= J(\theta) + \lambda \|\theta\|_2^2 \\ &= \frac{1}{N} \sum_{i=1}^N -\log p \left( y^{(i)} \mid \mathbf{x}^{(i)}, \theta \right) + \lambda \sum_{m=1}^M \theta_m^2 \end{aligned} \quad (\text{II.21})$$

| Choose form of $r(\boldsymbol{\theta})$ : | Notes                              |
|-------------------------------------------|------------------------------------|
| L2 norm: square-root of sum of squares    | → Small-valued parameters          |
| L1 norm: sum of absolute values           | → Zero-valued parameters           |
| L0 norm: count of non-zero values         | → Zero-values but hard to optimize |

### III. NEURAL NETWORKS (NNS)

## Neural Network



Example: Neural Network with 2 Hidden Layers and 2 Hidden Units

$$z_1^{(1)} = \sigma(\alpha_{11}^{(1)} x_1 + \alpha_{12}^{(1)} x_2 + \alpha_{13}^{(1)} x_3 + \alpha_{10}^{(1)})$$

$$z_2^{(1)} = \sigma(\alpha_{21}^{(1)} x_1 + \alpha_{22}^{(1)} x_2 + \alpha_{23}^{(1)} x_3 + \alpha_{20}^{(1)})$$

$$z_1^{(2)} = \sigma(\alpha_{11}^{(2)} z_1^{(1)} + \alpha_{12}^{(2)} z_2^{(1)} + \alpha_{10}^{(2)})$$

$$z_2^{(2)} = \sigma(\alpha_{21}^{(2)} z_1^{(1)} + \alpha_{22}^{(2)} z_2^{(1)} + \alpha_{20}^{(2)})$$

$$y = \sigma(\beta_1 z_1^{(2)} + \beta_2 z_2^{(2)} + \beta_0)$$

7

FIG. 10: A Neural Network with 2 Hidden Layers and 2 Hidden Units

## Deeper Networks

Q: How many layers should we use?

- **Theoretical answer:**

- A neural network with 1 hidden layer is a **universal function approximator**
- Cybenko (1989): For any continuous function  $g(\mathbf{x})$ , there exists a 1-hidden-layer neural net  $h_\theta(\mathbf{x})$  s.t.  $|h_\theta(\mathbf{x}) - g(\mathbf{x})| < \epsilon$  for all  $\mathbf{x}$ , assuming sigmoid activation functions

- **Empirical answer:**

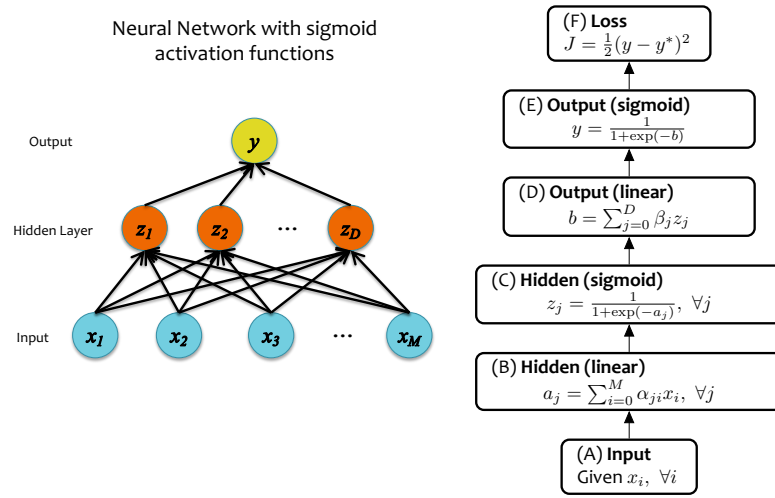
- Before 2006: “Deep networks (e.g. 3 or more hidden layers) are too hard to train”
- After 2006: “Deep networks are easier to train than shallow networks (e.g. 2 or fewer layers) for many problems”

Big caveat: You need to know and use the right tricks.

15

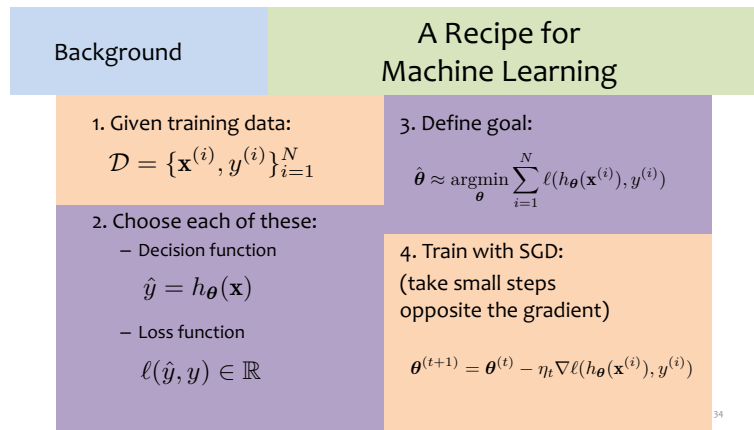
Deep networks (e.g. convolution networks): learn the increasingly higher levels of abstraction from data

# Activation Functions



23

FIG. 11: A Neural Network with Sigmoid Activation Functions



34

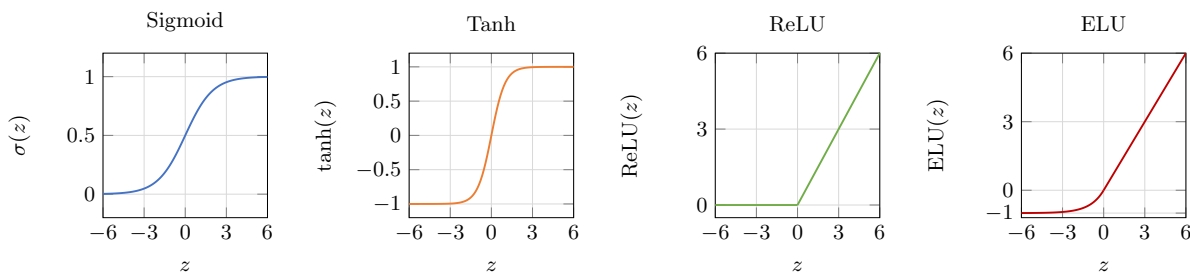


FIG. 12: Common Activation Functions

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \operatorname{ReLU}(z) = \max(0, z), \quad \operatorname{ELU}(z) = \begin{cases} z & z > 0 \\ \alpha(e^z - 1) & z \leq 0 \end{cases} \quad (\text{III.1})$$

### A. Objective Functions for Neural Networks

Objective 1. Binary Cross-Entropy: the same as the logistic regression loss function, which is used for binary classification problems. But this requires the output layer to have a sigmoid activation function to ensure the output  $y$  is between 0 and 1, which can be interpreted as probabilities.

$$J = \ell_{CE}(y, y^{(i)}) = - \left( y^{(i)} \log(y) + (1 - y^{(i)}) \log(1 - y) \right)$$

$$\frac{dJ}{dy} = - \left( y^{(i)} \frac{1}{y} + (1 - y^{(i)}) \frac{1}{y - 1} \right) \quad (\text{III.2})$$

This is also a softmax layer, which is a generalization of the sigmoid function for multi-class classification problems. The softmax function takes a vector of real-valued scores and converts it into a probability distribution over the classes.

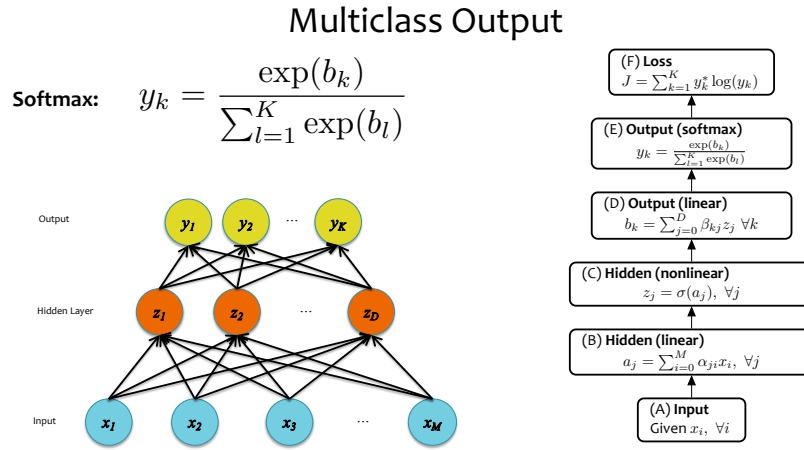
$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

$$\ell_{CE}(y, y^{(i)}) = - \sum_{k=1}^K y_k^{(i)} \log(\text{softmax}(z_k)) \quad (\text{III.3})$$

Objective 2. Quadratic Loss: used for regression problems, which measures the squared difference between the predicted output  $y$  and the true output  $y^{(i)}$ .

$$J = \ell_Q(y, y^{(i)}) = \frac{1}{2} (y - y^{(i)})^2$$

$$\frac{dJ}{dy} = y - y^{(i)} \quad (\text{III.4})$$



39

FIG. 13: A Neural Network with Multiclass Output

Finite Difference Method: The centered finite difference approximation is:

$$\frac{\partial}{\partial \theta_i} J(\theta) \approx \frac{J(\theta + \epsilon \cdot \mathbf{d}_i) - J(\theta - \epsilon \cdot \mathbf{d}_i)}{2\epsilon} \quad (\text{III.5})$$

where  $\mathbf{d}_i$  is a 1-hot vector consisting of all zeros except for the  $i$ th entry of  $\mathbf{d}_i$ , which has value 1.

| Methods  | Finite Difference Method                             | Symbolic Differentiation                                           | Automatic Differentiation - Reverse Mode                                                                                                                    | Automatic Differentiation - Forward Mode                                                                                                                    |
|----------|------------------------------------------------------|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Examples | $\frac{df}{dx} \approx \frac{f(x+h) - f(x)}{h}$      | $\frac{d}{dx} \sin(x) = \cos(x)$                                   | $\frac{df}{dx} = \frac{df}{du} \cdot \frac{du}{dx}$                                                                                                         | $\frac{df}{dx} = \frac{df}{du} \cdot \frac{du}{dx}$                                                                                                         |
| Pros     | Great for testing implementations of backpropagation | Yields easily interpretable derivatives                            | Computes partial derivatives of one output $f(\mathbf{x})_i$ with respect to all inputs $x_j$ in time polynomial in the computation time of $f(\mathbf{x})$ | Computes partial derivatives of all outputs $f(\mathbf{x})_i$ with respect to one input $x_j$ in time polynomial in the computation time of $f(\mathbf{x})$ |
| Cons     | Slow for high dimensional inputs / outputs           | Leads to exponential computation time if not carefully implemented | Slow for high dimensional outputs (e.g. vector-valued functions)                                                                                            | Slow for high dimensional inputs (e.g. vector-valued $\mathbf{x}$ )                                                                                         |

TABLE III: Methods for Computing Derivatives

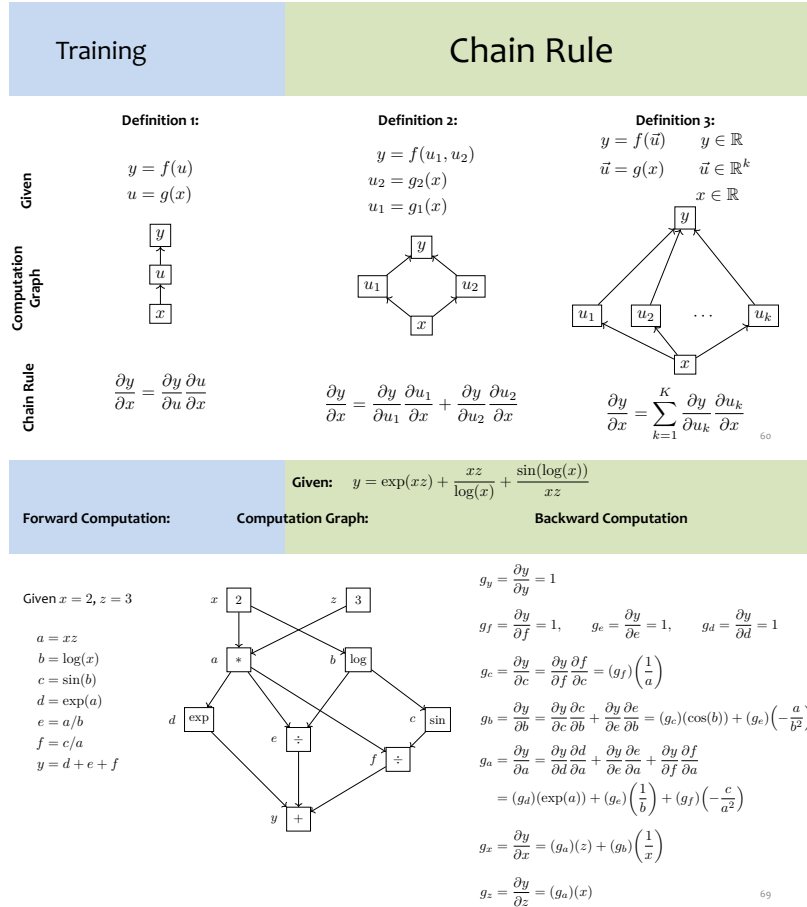


FIG. 14: Chain Rule and its Applications

## B. Backpropagation

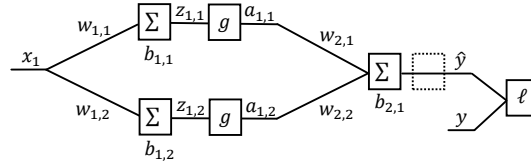
Updates for Backpropagation:

$$\begin{aligned}
 g_x &= \frac{\partial y}{\partial x} = \sum_{k=1}^K \frac{\partial y}{\partial u_k} \frac{\partial u_k}{\partial x} \\
 &= \sum_{k=1}^K g_{u_k} \frac{\partial u_k}{\partial x}
 \end{aligned} \tag{III.6}$$

Backprop is efficient because of reuse in the forward pass and the backward pass.

### Three-neuron Network

Network diagram



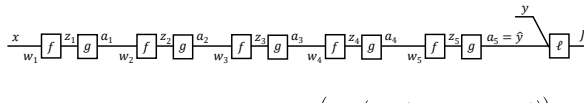
$$\begin{aligned}
 \hat{y} &= h_{\theta}(\mathbf{x}) = b_2 + \sum_j w_{2,j} a_{1,j} \\
 a_{1,j} &= g_{ReLU}(b_{1,j} + w_{1,j} x_1)
 \end{aligned}$$

FIG. 15: Three-Neuron Network

$\hat{y} = h_{\theta}(\mathbf{x}) = b_2 + \sum_j w_{2,j} a_{1,j}$ , for each of the three neurons in the hidden layer, they contain 1 weights and 1 biases,  $a_{1,j} = g_{ReLU}(b_{1,j} + w_{1,j} x_1)$  and the output neuron contains 3 weights and 1 bias, so there are a total of 10 parameters in this network.

#### Forward Pass

Width 1 deep network (no bias) (dumb but will help with calculus)

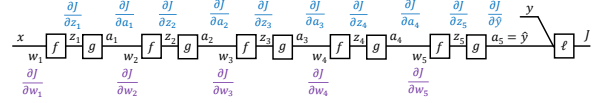


$$\begin{aligned}
 z_{\ell} &= f(w_{\ell}, a_{\ell-1}) = w_{\ell} \cdot a_{\ell-1} & \hat{y} &= h_{\theta}(\mathbf{x}) = g\left(w_5 \cdot g\left(w_4 \cdot g\left(w_3 \cdot g\left(w_2 \cdot g(w_1 \cdot x)\right)\right)\right)\right) \\
 a_{\ell} &= g(z_{\ell}) & &= g\left(f\left(g\left(f\left(g\left(f\left(g\left(f\left(g\left(f\left(g\left(f(x)\right)\right)\right)\right)\right)\right)\right)\right)\right)\right)\right)
 \end{aligned}$$

#### Backpropagation

$$\hat{y} = h_{\theta}(\mathbf{x}) = g\left(w_5 \cdot g\left(w_4 \cdot g\left(w_3 \cdot g\left(w_2 \cdot g(w_1 \cdot x)\right)\right)\right)\right)$$

Width 1 deep network (no bias) (dumb but will help with calculus)



Questions:

- Why do we need to compute all of the  $\partial J / \partial z_{\ell}$  and  $\partial J / \partial a_{\ell}$ ?
- Why "backwards"?
- How do we actually compute these?
- (What else do we need to learn to go beyond this useless network?)

FIG. 16: Forward pass and Backward pass

## C. Social Fairness

TABLE IV: Denominator Layout:  $\frac{\partial(\text{output})}{\partial(\text{input})} \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}$

|                                 |                                                                                         |                                                                                                           |                                                                                                         |
|---------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| $y = f(x)$                      | $x \in \mathbb{R}$                                                                      | $x \in \mathbb{R}^M$                                                                                      | $X \in \mathbb{R}^{M \times L}$                                                                         |
| $y \in \mathbb{R}$              | $\frac{\partial y}{\partial x} \in \mathbb{R}$                                          | $\frac{\partial y}{\partial \mathbf{x}} \in \mathbb{R}^{M \times 1}$<br>Input by output                   | $\frac{\partial y}{\partial X} \in \mathbb{R}^{M \times L}$<br>Size of input                            |
| $\mathbf{y} \in \mathbb{R}^N$   | $\frac{\partial \mathbf{y}}{\partial x} \in \mathbb{R}^{1 \times N}$<br>Input by output | Jacobian $\frac{\partial \mathbf{y}}{\partial \mathbf{x}} \in \mathbb{R}^{M \times N}$<br>Input by output | $\frac{\partial \mathbf{y}}{\partial X} \in \mathbb{R}^{ML \times N}$<br>Input size flattened by output |
| $Y \in \mathbb{R}^{N \times K}$ | $\frac{\partial Y}{\partial x} \in \mathbb{R}^{K \times N}$<br>Transpose of output      | $\frac{\partial Y}{\partial \mathbf{x}} \in \mathbb{R}^{M \times NK}$<br>Input size flattened by output   | $\frac{\partial Y}{\partial X} \in \mathbb{R}^{ML \times NK}$<br>Flattened input by flattened output    |

FIG. 17: Societal Implications of AI

|            |    | Predicted Label                    |                                    |                               |
|------------|----|------------------------------------|------------------------------------|-------------------------------|
|            |    | +1                                 | -1                                 |                               |
| True Label | +1 | True positive (TP)                 | False negative (FN)                | Total positives (P) = TP + FN |
|            | -1 | False positive (FP)                | True negative (TN)                 | Total negatives (N) = FP + TN |
|            |    | Predicted positives (PP) = TP + FP | Predicted negatives (PN) = FN + TN |                               |

2/26/25

15

|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Running Example | <ul style="list-style-type: none"> <li>Suppose you're an admissions officer for some program at CMU, deciding which applicants to admit</li> <li><math>X</math> are the non-protected features of an applicant (e.g., standardized test scores, GPA, etc...)</li> <li><math>A</math> is a protected feature (e.g. gender), usually categorical, i.e., <math>A \in \{a_1, \dots, a_C\}</math></li> <li><math>h(X, A) \in \{+1, -1\}</math> is your model's prediction, usually corresponding to some decision or action (e.g., <math>+1</math> = admit to CMU)</li> <li><math>Y \in \{+1, -1\}</math> is the true, underlying target variable, usually some latent or hidden state (e.g., <math>+1</math> = this applicant would be "successful" at CMU)</li> </ul> |
|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

2/26/25

20

## Three Definitions of Fairness

- **Independence (selection rate parity):**  $h(X, A) \perp A$ 
  - Proportion of accepted applicants is the same for all genders
  - Permits laziness/is susceptible to adversarial decisions
- **Separation (equality of FPR and FNR):**  $h(X, A) \perp A | Y$ 
  - All “good”/”bad” applicants are accepted with the same probability, regardless of gender
  - Perpetuates existing biases in the training data
- **Sufficiency (equality of PPV and NPV):**  $Y \perp A | h(X, A)$ 
  - For the purposes of predicting  $Y$ , the information contained in  $h(X, A)$  is “sufficient”,  $A$  becomes irrelevant

2/26/25

41

- False positive rate (FPR) =  $FP/N = FP / (FP + TN)$
- False negative rate (FNR) =  $FN/P = FN / (TP + FN)$
- Positive predictive value (PPV) =  $TP/PP = TP/(TP + FP)$
- Negative predictive value (NPV) =  $TN/PN = TN/(FN + TN)$

### D. Probably Approximately Correct (PAC) Learning

#### Model III.1: PAC/SLT Model for Supervised ML

- ▶ Set of possible inputs,  $\mathbf{x} \in \mathcal{X}$  (all possible patients)
- ▶ Set of possible outputs,  $y \in \mathcal{Y}$  (all possible diagnoses)
- ▶ Distribution over instances,  $p^*(\cdot)$
- ▶ Exists an unknown target function,  $c^* : \mathcal{X} \rightarrow \mathcal{Y}$  (the doctor’s brain)
- ▶ Set,  $\mathcal{H}$ , of candidate hypothesis functions,  $h : \mathcal{X} \rightarrow \mathcal{Y}$  (all possible decision trees)
- ▶ Learner is given  $N$  training examples  $D = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(N)}, y^{(N)})\}$  where  $\mathbf{x}^{(i)} \sim p^*(\cdot)$  and  $y^{(i)} = c^*(\mathbf{x}^{(i)})$  (history of patients and their diagnoses)
- ▶ Learner produces a hypothesis function,  $\hat{y} = h(\mathbf{x})$ , that best approximates unknown target function  $y = c^*(\mathbf{x})$  on the training data

♣ Two types of errors:

- ▶ True Error (aka. expected risk)

$$R(h) = P_{\mathbf{x} \sim p^*(\mathbf{x})} (c^*(\mathbf{x}) \neq h(\mathbf{x})) \quad (\text{III.7})$$

this is the probability that  $h$  misclassifies a random instance drawn from  $p^*$ . (which is always unknown)

| Cases    | Infinite $ \mathcal{H} $                                                             | Finite $ \mathcal{H} $                                               | Realizable Case                                                                 | Agnostic Case                                                                                            |
|----------|--------------------------------------------------------------------------------------|----------------------------------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Examples | the set of all decision trees of depth $D$ over binary feature vectors of length $M$ | the set of all neural networks with 1-hidden layer length $M$ inputs | the true hypothesis is definitely in the hypothesis space $c^* \in \mathcal{H}$ | the true hypothesis may not be in the hypothesis space $c^* \notin \mathcal{H}$ or $c^* \in \mathcal{H}$ |

► Empirical Error (aka. empirical risk)

$$\begin{aligned}
\hat{R}(h) &= P_{\mathbf{x} \sim \mathcal{S}}(c^*(\mathbf{x}) \neq h(\mathbf{x})) \\
&= \frac{1}{N} \sum_{i=1}^N \mathbb{1}(c^*(\mathbf{x}^{(i)}) \neq h(\mathbf{x}^{(i)})) \\
&= \frac{1}{N} \sum_{i=1}^N \mathbb{1}(y^{(i)} \neq h(\mathbf{x}^{(i)}))
\end{aligned} \tag{III.8}$$

where  $\mathcal{S} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}\}_{i=1}^N$  is the training data set, and  $\mathbf{x} \sim \mathcal{S}$  denotes that  $\mathbf{x}$  is sampled from the empirical distribution. This is the fraction of training examples that  $h$  misclassifies, which can be measured by the learner.

♣ The Goal: to choose a hypothesis  $h \in \mathcal{H}$  that has lowest error:

$$\begin{aligned}
&\textbf{expected risk minimizer} \text{ has lowest true error: } h^* = \underset{h \in \mathcal{H}}{\operatorname{argmin}} R(h) \\
&\textbf{empirical risk minimizer} \text{ has lowest training error: } h^* = \underset{h \in \mathcal{H}}{\operatorname{argmin}} \hat{R}(h)
\end{aligned} \tag{III.9}$$

♣ PAC Criterion: For any  $\epsilon > 0$  and  $\delta > 0$ , if the number of training examples  $N$  is sufficiently large, then with probability at least  $1 - \delta$ , the true error of the empirical risk minimizer is at most  $\epsilon$ , which means that our learner can learn a hypothesis that is approximately correct with high probability:

$$P(|R(h) - \hat{R}(h)| \leq \epsilon) \geq 1 - \delta \tag{III.10}$$

### Theorem III.1: Finite, Realizable Case

For a finite hypothesis set  $\mathcal{H}$  s.t.  $c^* \in \mathcal{H}$  and arbitrary distribution  $p^*$ , if the number of labeled training data points satisfies

$$N \geq \frac{1}{\epsilon} \left( \ln(|\mathcal{H}|) + \ln\left(\frac{1}{\delta}\right) \right) \tag{III.11}$$

then with probability at least  $1 - \delta$ , all  $h \in \mathcal{H}$  with  $\hat{R}(h) = 0$  have

$$R(h) \leq \epsilon.$$

Step 1. Call a hypothesis **bad** if:

$$R(h) > \epsilon.$$

We want to bound the probability that a bad  $h$  still has:

$$\hat{R}(h) = 0 \text{ (i.e., no mistakes on the sample).}$$

Step 2. Fix a bad hypothesis  $h$  with  $R(h) > \epsilon$ . Each sample is correct with probability at most:

$$1 - \epsilon \leq e^{-\epsilon}$$

For independent samples, the probability that  $h$  is correct on all  $N$  samples is at most:

$$P(\hat{R}(h) = 0) \leq (1 - \epsilon)^N \leq e^{-\epsilon N}$$

Step 3. Use the idea from **Union bound!**  $P(A \cup B) = P(A) + P(B) - P(A \cap B) \leq P(A) + P(B)$ , choose  $\delta$  to be the upper bound of the probability that there exists a bad hypothesis with zero training error:

$$\begin{aligned} \delta &\geq P(\exists h \in \mathcal{H} \text{ s.t. } R(h) > \epsilon \text{ and } \hat{R}(h) = 0) \\ &\geq \sum_{h \in \mathcal{H}} P(R(h) > \epsilon \text{ and } \hat{R}(h) = 0) \\ &\geq |\mathcal{H}| e^{-\epsilon N} = e^{\ln(|\mathcal{H}|) - \epsilon N} \end{aligned}$$

Thus, by rearranging, we proved Theorem III.1 for the finite, realizable case.

### Theorem III.2: Finite, Agnostic Case

For a finite hypothesis set  $\mathcal{H}$  and arbitrary distribution  $p^*$ , if the number of labelled training data points satisfies

$$N \geq \frac{1}{2\epsilon^2} \left( \ln(|\mathcal{H}|) + \ln\left(\frac{2}{\delta}\right) \right) \quad (\text{III.12})$$

then with probability at least  $1 - \delta$ , all  $h \in \mathcal{H}$  satisfy

$$|R(h) - \hat{R}(h)| \leq \epsilon$$

Step 1. For any fixed  $h \in \mathcal{H}$ , define the 0-1 loss:

$$Z_i = \mathbb{1} \left\{ h(x^{(i)}) \neq y^{(i)} \right\}, \quad Z_i \in [0, 1]$$

Then:

$$\hat{R}(h) = \frac{1}{N} \sum_{i=1}^N Z_i, \quad \mathbb{E}[Z_i] = R(h)$$

By Hoeffding's inequality:

$$P(|R(h) - \hat{R}(h)| > \epsilon) \leq 2 \exp(-2\epsilon^2 N)$$

Step 2. We want the bound to hold simultaneously for all  $h \in \mathcal{H}$ . Using the union bound:

$$P(\exists h \in \mathcal{H} : |R(h) - \hat{R}(h)| > \epsilon) \leq 2|\mathcal{H}| \exp(-2\epsilon^2 N)$$

by rearranging, we have proved Theorem III.2.

### Theorem III.3: Infinite, Realizable Case

For an infinite hypothesis set  $\mathcal{H}$  s.t.  $c^* \in \mathcal{H}$  and arbitrary distribution  $p^*$ , if the number of labeled training

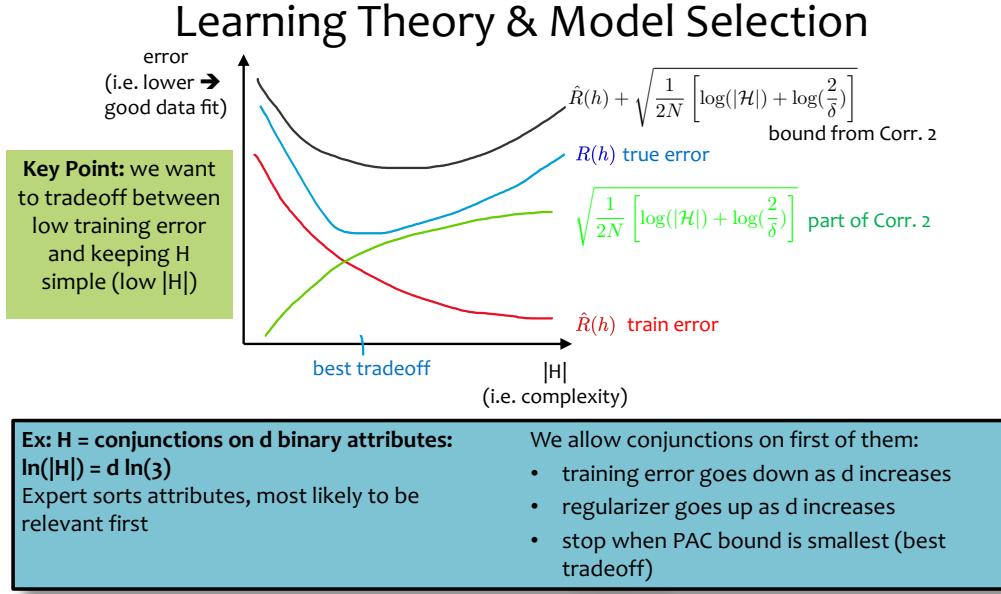


FIG. 18: Model Selection: a tradeoff between complexity and training error

data points satisfies

$$N = \mathcal{O} \left( \frac{1}{\epsilon} \left[ \text{VC}(\mathcal{H}) \log \left( \frac{1}{\epsilon} \right) + \log \left( \frac{1}{\delta} \right) \right] \right) \quad (\text{III.13})$$

labeled examples are sufficient so that with probability  $(1 - \delta)$  all  $h \in \mathcal{H}$  with  $\hat{R}(h) = 0$  have  $R(h) \leq \epsilon$ .

Step 1. Define:

$$\mathcal{H}_{\text{bad}} = \{h \in \mathcal{H} : R(h) > \epsilon\}.$$

We want to bound:

$$\mathbb{P} \left( \exists h \in \mathcal{H}_{\text{bad}} \text{ with } \hat{R}(h) = 0 \right).$$

Step 2. For any fixed  $h$  with  $R(h) > \epsilon$ :

$$\mathbb{P}(\hat{R}(h) = 0) \leq (1 - \epsilon)^N \leq e^{-\epsilon N}.$$

Step 3. Even though  $\mathcal{H}$  is infinite, on a sample of size  $N$  it induces at most can be found by VC growth,

$$\Pi_{\mathcal{H}}(N) \leq \left( \frac{eN}{d} \right)^d, \quad d = \text{VC}(\mathcal{H}) \quad \Rightarrow \quad \mathbb{P} \left( \exists h \in \mathcal{H}_{\text{bad}} \text{ with } \hat{R}(h) = 0 \right) \leq \left( \frac{eN}{d} \right)^d e^{-\epsilon N}.$$

Therefore,

$$\left( \frac{eN}{d} \right)^d e^{-\epsilon N} \leq \delta \quad \Rightarrow \quad N = \mathcal{O} \left( \frac{1}{\epsilon} \left[ d \log \left( \frac{1}{\epsilon} \right) + \log \left( \frac{1}{\delta} \right) \right] \right).$$

### Theorem III.4: Infinite, Agnostic Case

For an infinite hypothesis set  $\mathcal{H}$  s.t.  $c^* \in \mathcal{H}$  and arbitrary distribution  $p^*$ , if the number of labeled training data points satisfies

$$N = O\left(\frac{1}{\epsilon^2} \left[ \text{VC}(\mathcal{H}) + \log\left(\frac{1}{\delta}\right) \right]\right) \quad (\text{III.14})$$

labeled examples are sufficient so that with probability  $(1 - \delta)$  all  $h \in \mathcal{H}$  satisfy  $|R(h) - \hat{R}(h)| \leq \epsilon$ .

### E. VC Dimension

The **VC-dimension** (or Vapnik-Chervonenkis dimension) of  $\mathcal{H}$  is the cardinality of the largest set  $S$  such that  $\mathcal{H}$  can shatter  $S$ . We can write  $\text{VC}(\mathcal{H}) = d$ .

#### Proof III.1: VC-dimension Proof

Proof Technique: To prove that  $\text{VC}(\mathcal{H}) = d$  there are two steps:

Step 1. show that there exists a set of  $d$  points that can be shattered by  $\mathcal{H}$

$$\rightarrow \text{VC}(\mathcal{H}) \geq d$$

Step 2. show that there does NOT exist a set of  $d + 1$  points that can be shattered by  $\mathcal{H}$

$$\rightarrow \text{VC}(\mathcal{H}) < d + 1$$

#### 1. Why 4 Points in $\mathbb{R}^2$ Are Not Shattered by Linear Classifiers

Let  $\mathcal{H}$  be the class of linear classifiers (half-planes) in  $\mathbb{R}^2$ , and let  $S$  be a set of 4 points in general position (no three collinear).

a. *Definition (Shattering).* A set  $S$  is said to be *shattered* by  $\mathcal{H}$  if every possible labeling of  $S$  can be realized by some hypothesis  $h \in \mathcal{H}$ .

b. *Counting Labelings.* For  $|S| = 4$ , the total number of possible labelings is:

$$2^4 = 16.$$

To shatter  $S$ , all 16 labelings must be realizable by linear separators.

c. *Key Observation.* When the 4 points are in convex position (e.g., the vertices of a square), there exist labelings that are not linearly separable.

d. *Non-Separable Labelings (XOR Pattern).* Consider the labeling where opposite corners have the same label:

$$\begin{array}{cc} + & - \\ - & + \end{array}$$

In this configuration:

- The positive points lie on opposite corners.
- Any line that separates one positive point will misclassify the other.

Thus, no linear separator can realize this labeling.

The inverse labeling (swapping + and -) is also not linearly separable.

e. *Conclusion.* Out of the 16 possible labelings:

- 14 are realizable by linear classifiers,
- 2 (the XOR patterns) are not.

Therefore,  $S$  is *not shattered* by  $\mathcal{H}$ .

f. *Implication.* Since there exists a set of 4 points that cannot be shattered, the VC dimension of linear classifiers in  $\mathbb{R}^2$  is:

$$\text{VC}(\mathcal{H}) = 3.$$

From **Convex Hull**, two classes are linearly separable iff their convex hulls do not intersect.

| VC Dimension                                                                                                                                                                                                      | Shattering                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Proving VC-dimension requires us to show that there exists ( $\exists$ ) a dataset of size $d$ that can be shattered and that there does not exist ( $\nexists$ ) a dataset of size $d + 1$ that can be shattered | Proving that a particular dataset can be shattered requires us to show that for all ( $\forall$ ) labelings of the dataset, our hypothesis class contains a hypothesis that can correctly classify it |

## F. Maximum Likelihood Estimation (MLE)

### 1. Loss function

Given  $N$  independent, identically distributed (iid) samples  $D = \{x^{(1)}, x^{(2)}, \dots, x^{(N)}\}$  from a random variable  $X$ , the probability mass function (pmf,  $p(x|\theta)$  for discrete variables) or probability density function (pdf,  $f(x|\theta)$  for continuous variables). The likelihood function is defined as the joint probability of the observed data given the parameters  $\theta$ :

$$L(\theta) = p(x^{(1)} | \theta) p(x^{(2)} | \theta) \dots p(x^{(N)} | \theta) \quad (\text{III.15})$$

The log-likelihood function is:

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^N \log p(x^{(i)} | \theta) \quad (\text{III.16})$$

If  $Y$  is continuous with pdf  $f(y | \beta)$  and  $X$  is discrete with pmf  $p(x | y, \alpha)$ , then the likelihood function is:

$$L(\alpha, \beta) = p(x^{(1)} | y^{(1)}, \alpha) f(y^{(1)} | \beta) \dots p(x^{(N)} | y^{(N)}, \alpha) f(y^{(N)} | \beta) \quad (\text{III.17})$$

Examples:

- pdf of Exponential ( $\lambda$ ) :  $f(x) = \lambda e^{-\lambda x}$ . Suppose  $X_i \sim \text{Exponential}(\lambda)$  for  $1 \leq i \leq N$ . Find MLE for data  $\mathcal{D} = \{x^{(i)}\}_{i=1}^N$ .

$$\begin{aligned}
 \ell(\lambda) &= \sum_{i=1}^N \log f(x^{(i)}) \\
 &= \sum_{i=1}^N \log \left( \lambda \exp(-\lambda x^{(i)}) \right) \\
 &= \sum_{i=1}^N \left( \log(\lambda) - \lambda x^{(i)} \right) \\
 &= N \log(\lambda) - \lambda \sum_{i=1}^N x^{(i)}
 \end{aligned} \quad (\text{III.18})$$

Compute its first derivative and set it to zero:

$$\begin{aligned}
 \frac{d\ell(\lambda)}{d\lambda} &= \frac{d}{d\lambda} \left( N \log(\lambda) - \lambda \sum_{i=1}^N x^{(i)} \right) \\
 &= \frac{N}{\lambda} - \sum_{i=1}^N x^{(i)} = 0 \\
 \Rightarrow \lambda^{\text{MLE}} &= \frac{N}{\sum_{i=1}^N x^{(i)}}
 \end{aligned} \tag{III.19}$$

► Model:  $\mathbf{x}^{(i)} \sim \text{Bernoulli}(\phi)$  for  $i = 1, \dots, N$

$$\begin{aligned}
 p(x^{(i)} | \phi) &= \begin{cases} \phi & \text{if } x^{(i)} = 1 \\ (1 - \phi) & \text{if } x^{(i)} = 0 \end{cases} \\
 &= \phi^{x^{(i)}} (1 - \phi)^{1-x^{(i)}}
 \end{aligned} \tag{III.20}$$

Using  $N_0$  to denote the number of samples with  $x^{(i)} = 0$  and  $N_1$  to denote the number of samples with  $x^{(i)} = 1$ , the log-likelihood function is:

$$\begin{aligned}
 \ell_{\text{MLE}}(\phi) &= \log p(\mathcal{D} | \phi) \\
 &= \log \prod_{i=1}^N p(x^{(i)} | \phi) \\
 &= \log \prod_{i=1}^N \phi^{x^{(i)}} (1 - \phi)^{1-x^{(i)}} \\
 &= \log (\phi^{N_1} (1 - \phi)^{N_0}) \\
 &= N_1 \log(\phi) + N_0 \log(1 - \phi)
 \end{aligned} \tag{III.21}$$

And compute its first derivative and set it to zero:

$$\begin{aligned}
 \frac{\partial \ell_{\text{MLE}}(\phi)}{\partial \phi} &= \frac{\partial}{\partial \phi} (N_1 \log(\phi) + N_0 \log(1 - \phi)) \\
 &= \frac{N_1}{\phi} - \frac{N_0}{1 - \phi} = 0 \\
 \Rightarrow \phi^{\text{MLE}} &= \frac{N_1}{N_1 + N_0} = \frac{1}{N} \sum_{i=1}^N x^{(i)}
 \end{aligned} \tag{III.22}$$

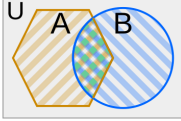
## G. Maximum A Posteriori Estimation (MAP)

### Theorem III.5: Bayes Theorem

For any two events  $A$  and  $B$  with  $P(B) \neq 0$ , the conditional probability of  $A$  given  $B$  is:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)} \tag{III.23}$$

$$\begin{aligned}
P(A) &= \frac{\text{yellow hexagon}}{\text{rectangle}}, & P(B|A) &= \frac{\text{yellow hexagon with blue dot}}{\text{yellow hexagon}} \\
P(B) &= \frac{\text{blue circle}}{\text{rectangle}}, & P(A|B) &= \frac{\text{yellow hexagon with blue dot}}{\text{blue circle}} \\
P(A) \cdot P(B|A) &= \frac{\text{yellow hexagon}}{\text{rectangle}} \times \frac{\text{yellow hexagon with blue dot}}{\text{yellow hexagon}} = \frac{\text{yellow hexagon with blue dot}}{\text{rectangle}} \\
P(B) \cdot P(A|B) &= \frac{\text{blue circle}}{\text{rectangle}} \times \frac{\text{yellow hexagon with blue dot}}{\text{blue circle}} = \frac{\text{yellow hexagon with blue dot}}{\text{rectangle}} \\
&= P(A) \cdot P(B|A), \text{ i.e.} \\
\end{aligned}$$



$$P(A|B) = \frac{P(A) \cdot P(B|A)}{P(B)}$$

$$P(B|A) = \frac{P(B) \cdot P(A|B)}{P(A)}$$

Bayes Theorem visual proof.

The MAP estimation objective is:

$$\underbrace{p(\boldsymbol{\theta} | \mathcal{D})}_{\text{Posterior}} = \frac{\overbrace{p(\mathcal{D} | \boldsymbol{\theta})}^{\text{Likelihood}} \overbrace{p(\boldsymbol{\theta})}^{\text{Prior}}}{p(\mathcal{D})} \quad (\text{III.24})$$

where  $p(\mathcal{D})$  is the evidence, which is a normalizing constant that does not depend on  $\boldsymbol{\theta}$ .

$$\begin{aligned}
\boldsymbol{\theta}_{MAP} &= \underset{\boldsymbol{\theta}}{\operatorname{argmax}} p(\boldsymbol{\theta} | \mathcal{D}) \\
&= \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \frac{p(\mathcal{D} | \boldsymbol{\theta}) p(\boldsymbol{\theta})}{p(\mathcal{D})} \\
&= \underset{\boldsymbol{\theta}}{\operatorname{argmax}} p(\mathcal{D} | \boldsymbol{\theta}) p(\boldsymbol{\theta}) \\
&= \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \underbrace{\log p(\mathcal{D} | \boldsymbol{\theta}) + \log p(\boldsymbol{\theta})}_{\ell_{MAP}(\boldsymbol{\theta})} \quad (\text{III.25})
\end{aligned}$$

| Posterior $p(\boldsymbol{\theta}   \mathcal{D})$ | Likelihood $p(\mathcal{D}   \boldsymbol{\theta})$ | Prior $p(\boldsymbol{\theta})$ | Conjugate? |
|--------------------------------------------------|---------------------------------------------------|--------------------------------|------------|
| Beta                                             | Bernoulli                                         | Beta                           | yes        |
| Dirichlet                                        | Multinomial                                       | Multinomial                    | yes        |
| Gaussian                                         | Guassian                                          | Guassian                       | yes        |
| Gamma                                            | Exponential                                       | Gamma                          | yes        |
| ??                                               | Multinomial                                       | Logistic Normal                | no         |

MAP of Beta-Bernoulli Model Model:  $\phi \sim \text{Beta}(\alpha, \beta)$ ,  $\mathbf{x}^{(i)} \sim \text{Bernoulli}(\phi)$  for  $i = 1, \dots, N$ , therefore, the

log-posterior is:

$$\begin{aligned}
 \ell_{\text{MAP}}(\phi) &= \log[p(\mathcal{D} \mid \phi)f(\phi \mid \alpha, \beta)] \\
 &= \log \left[ (\phi^{N_1}(1-\phi)^{N_0}) \left( \frac{1}{B(\alpha, \beta)} \phi^{(\alpha-1)}(1-\phi)^{(\beta-1)} \right) \right] \\
 &= \log \left[ \phi^{(N_1+\alpha-1)}(1-\phi)^{(N_0+\beta-1)} \frac{1}{B(\alpha, \beta)} \right] \\
 &= (N_1 + \alpha - 1) \log(\phi) + (N_0 + \beta - 1) \log(1 - \phi) - \log B(\alpha, \beta) \\
 &= N'_1 \log(\phi) + N'_0 \log(1 - \phi) - \log B(\alpha, \beta)
 \end{aligned} \tag{III.26}$$

where  $N'_1 = N_1 + \alpha - 1$  and  $N'_0 = N_0 + \beta - 1$ . Compute its first derivative and set it to zero:

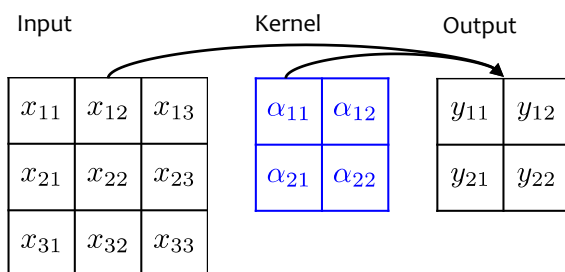
$$\frac{\partial \ell_{\text{MAP}}(\phi)}{\partial \phi} = \frac{N'_1}{\phi} - \frac{N'_0}{1-\phi} = 0 \quad \Rightarrow \quad \phi^{\text{MAP}} = \frac{N'_1}{N'_1 + N'_0} = \frac{N_1 + \alpha - 1}{N_1 + \alpha - 1 + N_0 + \beta - 1} = \frac{N_1 + \alpha - 1}{N + \alpha + \beta - 2} \tag{III.27}$$

## H. Convolution Neural Networks (CNNs) and Recurrent Neural Networks (RNNs)

### 2D Convolution

- Basic idea:
  - Pick a 2x2 matrix F of weights (called a kernel or convolution matrix)
  - Slide this over an image and compute the “inner product” (similarity) of F and the corresponding field of the image, and replace the pixel in the center of the field with the output of the inner product operation
- Key point:
  - Different convolutions extract different types of low-level “features” from an image
  - All that we need to vary to generate these different features is the weights of F

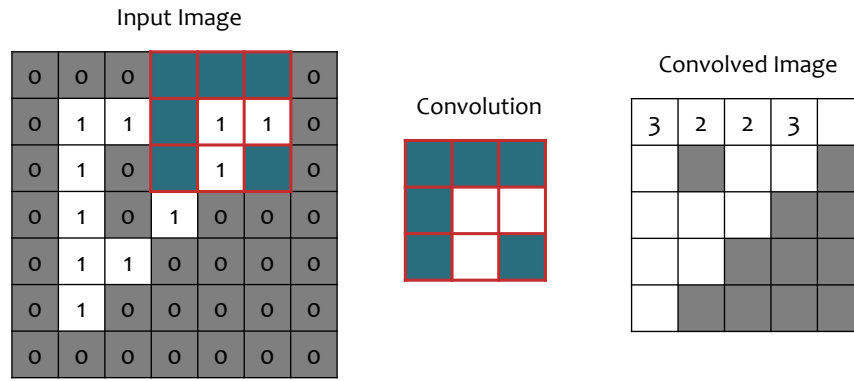
**Example: 1 input channel, 1 output channel**



$$\begin{aligned}
 y_{11} &= \alpha_{11}x_{11} + \alpha_{12}x_{12} + \alpha_{21}x_{21} + \alpha_{22}x_{22} + \alpha_0 \\
 y_{12} &= \alpha_{11}x_{12} + \alpha_{12}x_{13} + \alpha_{21}x_{22} + \alpha_{22}x_{23} + \alpha_0 \\
 y_{21} &= \alpha_{11}x_{21} + \alpha_{12}x_{22} + \alpha_{21}x_{31} + \alpha_{22}x_{32} + \alpha_0 \\
 y_{22} &= \alpha_{11}x_{22} + \alpha_{12}x_{23} + \alpha_{21}x_{32} + \alpha_{22}x_{33} + \alpha_0
 \end{aligned}$$

## 2D Convolution

- Pick a 2x2 matrix  $F$  of weights (called a kernel or convolution matrix)
- Slide this over an image and compute the “inner product” (similarity) of  $F$  and the corresponding field of the image, and replace the pixel in the center of the field with the output of the inner product operation



49

Output size:

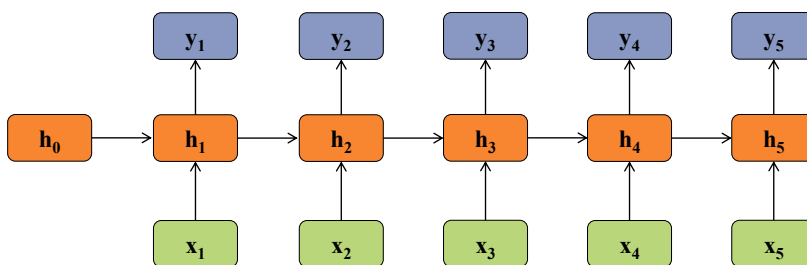
$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P - K}{S} \right\rfloor + 1 \quad (\text{III.28})$$

where  $W_{\text{in}}$  is the input width,  $P$  is the padding size,  $K$  is the convolutional kernel size, and  $S$  is the stride size.

## Recurrent Neural Networks (RNNs)

inputs:  $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$   
 hidden units:  $\mathbf{h} = (h_1, h_2, \dots, h_T), h_i \in \mathcal{R}^J$   
 outputs:  $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$   
 nonlinearity:  $\mathcal{H}$

Definition of the RNN:  
 $h_t = \mathcal{H}(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$   
 $y_t = W_{hy}h_t + b_y$



This form of RNN is called an **Elman Network**

11

By unrolling the RNN through time, we can share parameters and accommodate **arbitrary length** input/output pairs, which allows RNNs to deal with time-series data and sequential data.

inputs:  $\mathbf{x} = (x_1, x_2, \dots, x_T), x_i \in \mathcal{R}^I$ ; hidden units:  $\mathbf{h} = (h_1, h_2, \dots, h_T), h_i \in \mathcal{R}^J$ ; outputs:  $\mathbf{y} = (y_1, y_2, \dots, y_T), y_i \in \mathcal{R}^K$ ; nonlinearity:  $\mathcal{H}$

$$\begin{aligned} h_t &= \mathcal{H}(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \\ y_t &= W_{hy}h_t + b_y \end{aligned} \quad (\text{III.29})$$

Since the parameters are shared across time, the total number of trainable parameters is independent of the sequence length  $T$ . Specifically,

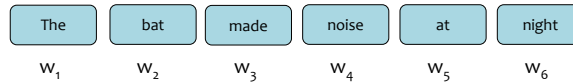
$$\begin{aligned} W_{xh} &\in \mathcal{R}^{J \times I}, & W_{hh} &\in \mathcal{R}^{J \times J}, & b_h &\in \mathcal{R}^J, \\ W_{hy} &\in \mathcal{R}^{K \times J}, & b_y &\in \mathcal{R}^K. \end{aligned}$$

Therefore, the total number of parameters is

$$JI + J^2 + J + KJ + K. \quad (\text{III.30})$$

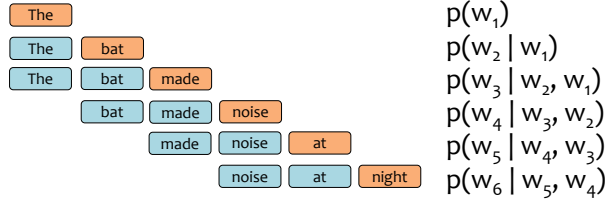
## n-Gram Language Model

**Question:** How can we **define** a probability distribution over a sequence of length  $T$ ?



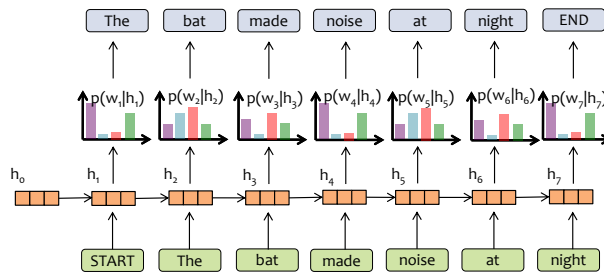
$$\text{n-Gram Model (n=3)} \quad p(w_1, w_2, \dots, w_T) = \prod_{t=1}^T p(w_t | w_{t-1}, w_{t-2})$$

$$p(w_1, w_2, w_3, \dots, w_6) =$$



27

## RNN Language Model



$$p(w_1, w_2, w_3, \dots, w_T) = p(w_1 | h_1) p(w_2 | h_2) \dots p(w_T | h_T)$$

50

**Algorithm 10** Elman RNN

---

```

1: procedure FORWARD($x_{1:T}, W_{ah}, W_{ax}, b_a, W_{yh}, b_y$)
2: Initialize the hidden state h_0 to zeros
3: for $t = 1$ to T do
4: Receive input data at time step t : x_t
5: Compute the hidden state update:
6: $a_t \leftarrow W_{ah}h_{t-1} + W_{ax}x_t + b_a$
7: $h_t \leftarrow \sigma(a_t)$
8: Compute the output at time step t :
9: $y_t \leftarrow \text{softmax}(W_{yh}h_t + b_y)$

```

---

**Algorithm 11** Elman RNN + Loss

---

```

1: procedure FORWARD($x_{1:T}, y_{1:T}^*, W_{ah}, W_{ax}, b_a, W_{yh}, b_y$)
2: Initialize the hidden state h_0 to zeros
3: for $t = 1$ to T do
4: Receive input data at time step t : x_t
5: Compute the hidden state update:
6: $a_t \leftarrow W_{ah}h_{t-1} + W_{ax}x_t + b_a$
7: $h_t \leftarrow \sigma(a_t)$
8: Compute the output at time step t :
9: $y_t \leftarrow \text{softmax}(W_{yh}h_t + b_y)$
10: Compute the cross-entropy loss at time step t :
11: $\ell_t \leftarrow -\sum_{k=1}^K (y_t^*)_k \log((y_t)_k)$
12: Compute the total loss:
13: $\ell \leftarrow \sum_{t=1}^T \ell_t$

```

---

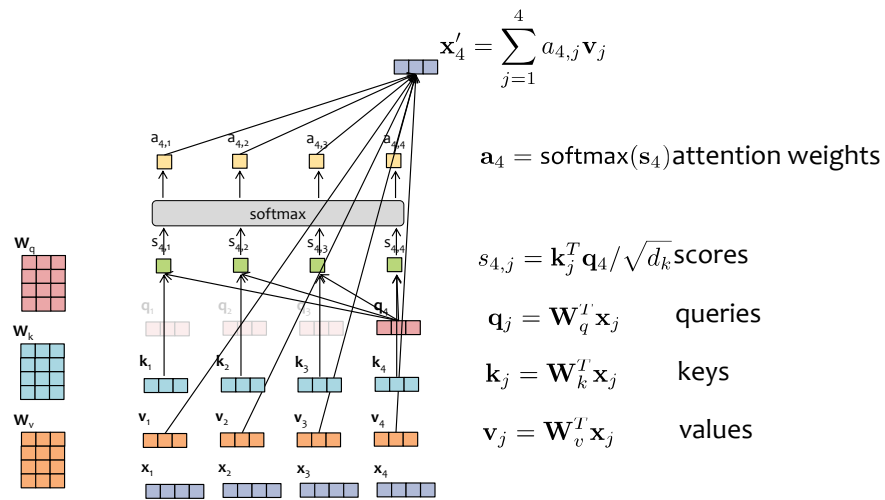
1. *Representative LLM Scales*

The original slide mixed tokens, words, and rumor-based parameter estimates. A cleaner comparison is to keep only **publicly reported** scale information and to mark undisclosed values explicitly. For this reason, the training-data column below uses the original unit reported by each creator.

TABLE V: Representative large language models with publicly reported scale information, updated through April 29, 2026.

| Model                     | Organization    | Release | Training data (publicly reported)        | Parameters (publicly reported) |
|---------------------------|-----------------|---------|------------------------------------------|--------------------------------|
| GPT-2                     | OpenAI          | 2019    | WebText, 40 GB text from 8M web pages    | 1.5B                           |
| GPT-3                     | OpenAI          | 2020    | 300B tokens                              | 175B                           |
| LaMDA                     | Google Research | 2022    | 1.56T words (2.81T SentencePiece tokens) | up to 137B                     |
| PaLM                      | Google Research | 2022    | 780B tokens                              | 540B                           |
| Chinchilla                | DeepMind        | 2022    | $\sim 1.3$ T tokens                      | 70B                            |
| Llama (65B)               | Meta            | 2023    | 1.4T tokens                              | 65B                            |
| Llama 2                   | Meta            | 2023    | 2T tokens                                | 70B                            |
| GPT-4                     | OpenAI          | 2023    | Not publicly disclosed                   | Not publicly disclosed         |
| Gemini 1.0 Ultra          | Google DeepMind | 2023    | Not publicly disclosed                   | Not publicly disclosed         |
| Llama 3.1 405B            | Meta            | 2024    | > 15T tokens                             | 405B                           |
| DeepSeek-V3               | DeepSeek        | 2024    | 14.8T tokens                             | 671B total, 37B active (MoE)   |
| GPT-4.1                   | OpenAI          | 2025    | Not publicly disclosed                   | Not publicly disclosed         |
| Gemini 2.5 Pro            | Google DeepMind | 2025    | Not publicly disclosed                   | Not publicly disclosed         |
| Claude Opus 4.7           | Anthropic       | 2026    | Not publicly disclosed                   | Not publicly disclosed         |
| DeepSeek-V4-Pro (Preview) | DeepSeek        | 2026    | Not publicly disclosed                   | 1.6T total, 49B active (MoE)   |

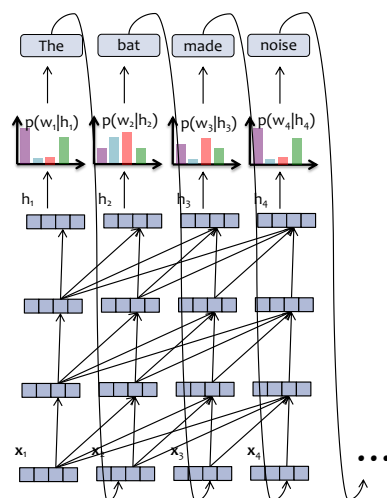
## Scaled Dot-Product Attention



96

## I. Transformer

### Transformer Language Model



The language model part is just like an RNN-LM!

**Each layer** of a Transformer LM consists of several **sublayers**:

1. attention
2. feed-forward neural network
3. layer normalization
4. residual connections

11

## Layer Normalization

- **The Problem:** **internal covariate shift** occurs during training of a deep network when a small change in the low layers amplifies into a large change in the high layers
- **One Solution:** **Layer normalization** normalizes each layer and learns elementwise gain/bias
- Such normalization allows for higher learning rates (for **faster convergence**) without issues of diverging gradients

Given input  $\mathbf{a} \in \mathbb{R}^K$ , LayerNorm computes output  $\mathbf{b} \in \mathbb{R}^K$ :

$$\mathbf{b} = \gamma \odot \frac{\mathbf{a} - \mu}{\sigma} \oplus \beta$$

where we have mean  $\mu = \frac{1}{K} \sum_{k=1}^K a_k$ ,  
 standard deviation  $\sigma = \sqrt{\frac{1}{K} \sum_{k=1}^K (a_k - \mu)^2}$ ,  
 and parameters  $\gamma \in \mathbb{R}^K, \beta \in \mathbb{R}^K$ .

$\odot$  and  $\oplus$  denote elementwise multiplication and addition.

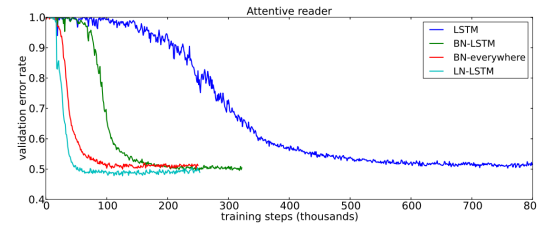


Figure from <https://arxiv.org/pdf/1607.06450.pdf>

12

Why are **residual connections** helpful?

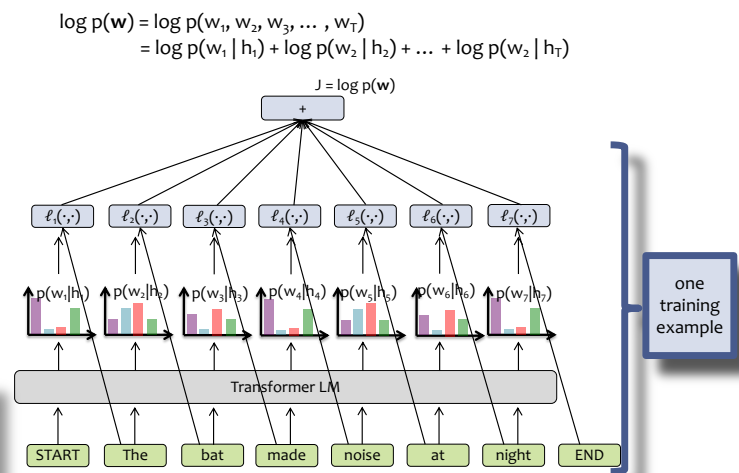
Instead of  $f(\mathbf{a})$  having to learn a full transformation of  $\mathbf{a}$ ,  $f(\mathbf{a})$  only needs to learn an additive modification of  $\mathbf{a}$  (i.e. the residual).

## Learning a Transformer LM

- Each training example is a sequence (e.g. sentence), so we have training data  $D = \{\mathbf{w}^{(1)}, \mathbf{w}^{(2)}, \dots, \mathbf{w}^{(N)}\}$
- The objective function for a Deep LM (e.g. RNN-LM or Transformer-LM) is typically the log-likelihood of the training examples:  

$$J(\theta) = \sum_i \log p_{\theta}(\mathbf{w}^{(i)})$$
- We train by mini-batch SGD (or your favorite flavor of mini-batch SGD)

Training a Transformer-LM is the same, except we swap in a different deep language model.



51

## IV. REINFORCEMENT LEARNING

### A. Reinforcement Learning

## Few-shot Learning with LLMs

Suppose you have...

- a dataset  $D = \{(x_i, y_i)\}_{i=1}^N$  and  $N$  is rather small (i.e. few-shot setting)
  - a very large (billions of parameters) pre-trained language model
- There are two ways to “learn”

This section!

#### Option A: Supervised fine-tuning

- **Definition:** fine-tune the LLM on the training data using...
  - a standard supervised objective
  - backpropagation to compute gradients
  - your favorite optimizer (e.g. Adam)
- **Pro:** fits into the standard ML recipe
- **Pro:** still works if  $N$  is large
- **Con:** backpropagation requires  $\sim 3x$  the memory and computation time as the forward computation
- **Con:** you might not have access to the model weights at all (e.g. because the model is proprietary)

#### Option B: In-context learning

- **Definition:**
  1. feed training examples to the LLM as a prompt
  2. allow the LLM to infer patterns in the training examples during inference (i.e. decoding)
  3. take the output of the LLM following the prompt as its prediction
- **Con:** the prompt may be very long and Transformer LMs require  $O(N^2)$  time/space where  $N$  = length of context
- **Pro:** no backpropagation required and only one pass through the training data
- **Pro:** does not require model weights, only API access

32

Markov Decision Processes  
Standard expectimax:

$$V(s) = \max_a \sum_{s'} P(s' | s, a) V(s') \quad (\text{IV.1})$$

Bellman equations:

$$V^*(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^*(s')] \quad (\text{IV.2})$$

Value iteration:

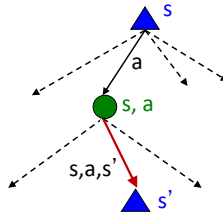
$$V_{k+1}(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_k(s')], \quad \forall s \quad (\text{IV.3})$$

Recursive Expectimax (with rewards!)

$$V(s) = \max_a \sum_{s'} P(s'|s, a) V(s')$$

With rewards:

$$V(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$$



where  $s$  is the state,  $a$  is the action,  $s'$  is the next state,  $P(s'|s, a)$  is the transition probability,  $R(s, a, s')$  is the reward function, and  $\gamma$  is the discount factor.  $V(s)$  is the value function, which represents the expected cumulative reward starting from state  $s$  and following the optimal policy thereafter. Policy is defined as  $\pi(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')]$ .

## Two Methods for Solving MDPs

### Value iteration + policy extraction

- **Step 1: Value iteration:**

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')], \quad \forall s \text{ until convergence}$$

- **Step 2: Policy extraction:**

$$\pi_V(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')], \quad \forall s$$

### Policy iteration

- **Step 1: Policy evaluation:**

$$V_{k+1}^\pi(s) = \sum_{s'} P(s'|s, \pi(s)) [R(s, \pi(s), s') + \gamma V_k^\pi(s')], \quad \forall s \text{ until convergence}$$

- **Step 2: Policy improvement:**

$$\pi_{new}(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^{\pi_{old}}(s')], \quad \forall s$$

- **Repeat** steps until policy converges

## MDP Notation

Standard expectimax:  $V(s) = \max_a \sum_{s'} P(s'|s, a) V(s')$

Bellman equations:  $V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')]$

Value iteration:  $V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')], \quad \forall s$

Q-iteration:  $Q_{k+1}(s, a) = \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma \max_{a'} Q_k(s', a')], \quad \forall s, a$

Policy extraction:  $\pi_V(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')], \quad \forall s$

Policy evaluation:  $V_{k+1}^\pi(s) = \sum_{s'} P(s'|s, \pi(s)) [R(s, \pi(s), s') + \gamma V_k^\pi(s')], \quad \forall s$

Policy improvement:  $\pi_{new}(s) = \operatorname{argmax}_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^{\pi_{old}}(s')], \quad \forall s$

## B. Policy Gradient

How to Learn  $V^\pi$  given  $\pi$  ?

- **Problem:** Large state space (e.g., high-resolution images):  $V(s)$  cannot be represented as a table.
- **Solution:**
  - Use a function approximator (e.g., a neural network) to represent  $V(s)$  as  $V_\phi(s)$ , where  $\phi$  are the parameters.
  - Estimate  $V^\pi(s)$  via Monte Carlo: sample trajectories from  $\pi$  and average the returns.
  - Update  $\phi$  via gradient descent to minimize MSE between  $V_\phi(s)$  and the Monte Carlo estimates of  $V^\pi(s)$ .

Example function approximators:

- **Linear function approximation:**  
 $V_\phi(s) = \phi^T \mathbf{x}(s)$ , where  $\mathbf{x}(s)$  is a feature vector representation of state  $s$ .
- **One-hidden layer neural network:**  
 $V_\phi(s) = \mathbf{w}^T \sigma(\mathbf{W}\mathbf{x}(s) + \mathbf{b})$ , where  $\mathbf{W}, \mathbf{b}, \mathbf{w}$  are the parameters of the neural network,  $\mathbf{x}(s)$  is a feature vector representation of state  $s$ , and  $\sigma$  is a non-linear activation function (e.g., ReLU).

31

## Algorithm 12 On-Policy Prediction with Function Approximation

```

1: procedure MONTECARLOPREDICTIONFA(π, η, γ)
2: Initialize value parameters ϕ randomly
3: while not converged do
4: Sample a trajectory $\tau = [(s_0, a_0, r_0), \dots, (s_T, a_T, r_T)]$ using π
5: $g \leftarrow \sum_{t=0}^T \nabla_\phi (V_\phi(s_t) - \bar{R}_t)^2$
6: where $\bar{R}_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$
7: $\phi \leftarrow \phi - \eta g$
8: return V_ϕ

```

## Theorem IV.1: Policy Gradient Identity

Let

$$p_\theta(\tau) = \prod_{t=0}^T \pi_\theta(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

be the probability of a trajectory  $\tau$  under policy  $\pi_\theta$ , where  $s_0$  is the initial state. Then

$$\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] \\
&= \nabla_\theta \sum_{\tau} [R(\tau) p_\theta(\tau)] \\
&= \sum_{\tau} [R(\tau) \nabla_\theta p_\theta(\tau)] \\
&= \sum_{\tau} \left[ R(\tau) p_\theta(\tau) \frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)} \right] \\
&= \sum_{\tau} [R(\tau) p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)] \\
&= \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau) \nabla_\theta \log p_\theta(\tau)].
\end{aligned} \tag{IV.4}$$

### Theorem IV.2: Policy Gradient Theorem

Let

$$p_\theta(\tau) = \prod_{t=0}^T \pi_\theta(a_t | s_t) p(s_{t+1} | s_t, a_t).$$

Then

$$\begin{aligned} \nabla_\theta J(\theta) &= \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau) \nabla_\theta \log p_\theta(\tau)] \\ &= \mathbb{E}_{\tau \sim \pi_\theta} \left[ R(\tau) \nabla_\theta \sum_{t=0}^T (\log \pi_\theta(a_t | s_t) + \log p(s_{t+1} | s_t, a_t)) \right] \\ &= \mathbb{E}_{\tau \sim \pi_\theta} \left[ R(\tau) \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \right] \\ &= \mathbb{E}_{\tau \sim \pi_\theta} \left[ \sum_{t=0}^T \bar{R}_t \nabla_\theta \log \pi_\theta(a_t | s_t) \right], \end{aligned} \tag{IV.5}$$

where

$$\bar{R}_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'} \tag{IV.6}$$

is the discounted return from time step  $t$ . The last equality follows because the reward terms prior to time  $t$  have expectation zero when multiplied by  $\nabla_\theta \log \pi_\theta(a_t | s_t)$ .

### Theorem IV.3: Monte Carlo Policy Gradient Estimator

We can obtain an unbiased estimate of the policy gradient by sampling a trajectory  $\tau$  from policy  $\pi_\theta$  and computing

$$g = \sum_{t=0}^T \bar{R}_t \nabla_\theta \log \pi_\theta(a_t | s_t), \tag{IV.7}$$

where

$$\bar{R}_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}. \tag{IV.8}$$

That is,

$$\mathbb{E}_{\tau \sim \pi_\theta} [g] = \nabla J(\theta). \tag{IV.9}$$

---

**Algorithm 13** REINFORCE
 

---

```

1: procedure REINFORCE(η, γ)
2: Initialize policy parameters θ randomly
3: while not converged do
4: Sample a trajectory $\tau = [(s_0, a_0, r_0), \dots, (s_T, a_T, r_T)]$ from π_θ
5: $g \leftarrow \sum_{t=1}^T \bar{R}_t \nabla_\theta \log \pi_\theta(a_t | s_t)$
6: where $\bar{R}_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$
7: $\theta \leftarrow \theta + \eta g$
8: return π_θ

```

---

**Policy Gradient Example** Assume the current policy parameters are

$$w = 0, \quad b = 0.$$

and the sampled trajectory is

$$\tau = [(0, R, -0.5), (1, L, -0.5), (0, R, -0.5), (1, R, 3)].$$

**Step 1.** Since  $w = 0$  and  $b = 0$ , we have  $wx + b = 0$  for every state  $x$ . Hence

$$\pi_{w,b}(R | x) = \pi_{w,b}(L | x) = \frac{1}{2}.$$

**Step 2.** With  $\gamma = 0.9$ , the discounted returns are

$$\bar{R}_3 = 3, \quad \bar{R}_2 = 2.2, \quad \bar{R}_1 = 1.48, \quad \bar{R}_0 = 0.832.$$

**Step 3.** Since the policy is uniform,

$$\nabla_\theta \log \pi(R | x) = \frac{1}{2} \begin{bmatrix} x \\ 1 \end{bmatrix}, \quad \nabla_\theta \log \pi(L | x) = \frac{1}{2} \begin{bmatrix} -x \\ -1 \end{bmatrix}.$$

**Step 4.** The REINFORCE contributions at each timestep are

$$\begin{aligned} t = 0 : 0.832 \cdot \frac{1}{2} \begin{bmatrix} 0 \\ 1 \end{bmatrix} &= \begin{bmatrix} 0 \\ 0.416 \end{bmatrix}, \\ t = 1 : 1.48 \cdot \frac{1}{2} \begin{bmatrix} -1 \\ -1 \end{bmatrix} &= \begin{bmatrix} -0.74 \\ -0.74 \end{bmatrix}, \\ t = 2 : 2.2 \cdot \frac{1}{2} \begin{bmatrix} 0 \\ 1 \end{bmatrix} &= \begin{bmatrix} 0 \\ 1.1 \end{bmatrix}, \\ t = 3 : 3 \cdot \frac{1}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix} &= \begin{bmatrix} 1.5 \\ 1.5 \end{bmatrix}. \end{aligned}$$

**Step 5.** Summing the four terms gives

$$g = \begin{bmatrix} 0.76 \\ 2.276 \end{bmatrix}.$$

Therefore the REINFORCE update

$$\theta \leftarrow \theta + \eta g$$

increases both  $w$  and  $b$ , so  $\pi(R \mid 1)$  increases. In words, the updated policy becomes more likely to move right at  $x = 1$ .

---

**Algorithm 14** REINFORCE with Baseline

---

```

1: procedure REINFORCEWITHBASELINE($\eta_\theta, \eta_\phi, \gamma$)
2: Initialize policy parameters θ and baseline parameters ϕ randomly
3: while not converged do
4: Sample a trajectory $\tau = [(s_0, a_0, r_0), \dots, (s_T, a_T, r_T)]$ from π_θ
5: $g_\theta \leftarrow \sum_{t=1}^T (\bar{R}_t - V_\phi(s_t)) \nabla_\theta \log \pi_\theta(a_t \mid s_t)$
6: where $\bar{R}_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$
7: $g_\phi \leftarrow \sum_{t=1}^T \nabla_\phi (V_\phi(s_t) - \bar{R}_t)^2$
8: $\theta \leftarrow \theta + \eta_\theta g_\theta$
9: $\phi \leftarrow \phi - \eta_\phi g_\phi$
10: return π_θ, V_ϕ

```

---

To reduce variance, we introduce a baseline function  $b(s)$  by replacing  $R_t$  with  $(R_t - b(s_t))$ .  $b(s)$  can be any function of the state and  $g$  will still be an unbiased estimator of  $\nabla J(\theta)$ . The value function  $V^\pi(s)$  is a common choice for  $b(s)$

---

**Algorithm 15** Actor-Critic

---

```

1: procedure ACTORCRITIC($\eta_\theta, \eta_\phi, \gamma$)
2: Initialize policy parameters θ and value parameters ϕ randomly
3: while not converged do
4: Collect a trajectory $\tau = [(s_0, a_0, r_0), \dots, (s_T, a_T, r_T)]$ from π_θ
5: $g_\theta \leftarrow \sum_{t=1}^T \hat{Q}_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$
6: where $\hat{Q}_t = r_t + \gamma V_\phi(s_{t+1})$
7: $g_\phi \leftarrow \sum_{t=1}^T \nabla_\phi (V_\phi(s_t) - \hat{Q}_t)^2$
8: $\theta \leftarrow \theta + \eta_\theta g_\theta$
9: $\phi \leftarrow \phi - \eta_\phi g_\phi$
10: return π_θ, V_ϕ

```

---

the policy is the actor the value function is the critic

---

**Algorithm 16** Advantage Actor-Critic (A2C)

---

```

1: procedure A2C($\eta_\theta, \eta_\phi, \gamma$)
2: Initialize policy parameters θ and value parameters ϕ randomly
3: while not converged do
4: Collect a trajectory $\tau = [(s_0, a_0, r_0), \dots, (s_T, a_T, r_T)]$ from π_θ
5: $g_\theta \leftarrow \sum_{t=1}^T \hat{A}_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$
6: where $\hat{A}_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$
7: $g_\phi \leftarrow \sum_{t=1}^T \nabla_\phi (r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t))^2$
8: $\theta \leftarrow \theta + \eta_\theta g_\theta$
9: $\phi \leftarrow \phi - \eta_\phi g_\phi$
10: return π_θ, V_ϕ

```

---

$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$  is called the advantage of  $a$  in  $s$ . large  $A^\pi(s, a)$  means it's advantageous to take  $a$  in  $s$  (vs. following  $\pi$ )

**Algorithm 17** A2C- $n$ Step

---

```

1: procedure A2C-NSTEP($\eta_\theta, \eta_\phi, \gamma, n$)
2: Initialize policy parameters θ and value parameters ϕ randomly
3: while not converged do
4: Collect a trajectory $\tau = [(s_0, a_0, r_0), \dots, (s_T, a_T, r_T)]$ from π_θ
5: $g_\theta \leftarrow \sum_{t=1}^T \hat{A}_t^{(n)} \nabla_\theta \log \pi_\theta(a_t | s_t)$
6: where $\hat{A}_t^{(n)} = \hat{R}_t^{(n)} - V_\phi(s_t)$
7: where $\hat{R}_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V_\phi(s_{t+n})$
8: $g_\phi \leftarrow \sum_{t=1}^T \nabla_\phi \left(\hat{R}_t^{(n)} - V_\phi(s_t) \right)^2$
9: $\theta \leftarrow \theta + \eta_\theta g_\theta$
10: $\phi \leftarrow \phi - \eta_\phi g_\phi$
11: return π_θ, V_ϕ

```

---

To reduce bias, we can use an  $n$ -step return to estimate the advantage function.

## Comparison of Policy Gradient Methods

1) All of the algorithms compute a gradient of the form:

$$\mathbf{g}_\theta = \sum_{t=1}^T \tilde{G}_t \nabla_\theta \log \pi_\theta(a_t | s_t)$$

where  $\tilde{G}_t$  is an estimate of the expected discounted future reward from time step  $t$

2) Some use a  $V_\phi$ , and some do not

| Algorithm               | $\tilde{G}_t$                                            | Estimate $V_\phi$ ? |
|-------------------------|----------------------------------------------------------|---------------------|
| REINFORCE               | $\bar{R}_t$                                              | ✗                   |
| REINFORCE with Baseline | $(\bar{R}_t - V_\phi(s_t))$                              | ✓                   |
| Actor-Critic            | $\hat{Q}_t = r_t + \gamma V_\phi(s_{t+1})$               | ✓                   |
| Advantage Actor-Critic  | $\hat{A}_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ | ✓                   |

## V. LEARNING PARADIGMS

### Learning Paradigms

| Paradigm                | Data                                                                                                             |
|-------------------------|------------------------------------------------------------------------------------------------------------------|
| Supervised              | $\mathcal{D} = \{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^N$ $\mathbf{x} \sim p^*(\cdot)$ and $y = c^*(\cdot)$          |
| ↔ Regression            | $y^{(i)} \in \mathbb{R}$                                                                                         |
| ↔ Classification        | $y^{(i)} \in \{1, \dots, K\}$                                                                                    |
| ↔ Binary classification | $y^{(i)} \in \{+1, -1\}$                                                                                         |
| ↔ Structured Prediction | $\mathbf{y}^{(i)}$ is a vector                                                                                   |
| Unsupervised            | $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^N$ $\mathbf{x} \sim p^*(\cdot)$                                        |
| Semi-supervised         | $\mathcal{D} = \{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^{N_1} \cup \{\mathbf{x}^{(j)}\}_{j=1}^{N_2}$                  |
| Online                  | $\mathcal{D} = \{(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), (\mathbf{x}^{(3)}, y^{(3)}), \dots\}$ |
| Active Learning         | $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^N$ and can query $y^{(i)} = c^*(\cdot)$ at a cost                      |
| Imitation Learning      | $\mathcal{D} = \{(s^{(1)}, a^{(1)}), (s^{(2)}, a^{(2)}), \dots\}$                                                |
| Reinforcement Learning  | $\mathcal{D} = \{(s^{(1)}, a^{(1)}, r^{(1)}), (s^{(2)}, a^{(2)}, r^{(2)}), \dots\}$                              |

42

### A. Recommender Systems

#### Different Approaches

##### Item-based (*Content filtering*)

- Features about each item
- Given an item, other “close” items have similar values
- e.g. Pandora.com, music genome project

##### User-based

- Features about each user
- Given a user, other “close” users have similar preferences
- *Market segmentation*

##### Learning user-item relationship

- Can be done without features on either user or item
- Collaborative filtering techniques

Two methods of collaborative filtering:

- Neighborhood-based methods: predict a user’s rating for an item based on the ratings of similar users or similar items.
- Latent Factor methods: Assume that both items and users can be mapped to the same feature space.

Optimization: Objective function using only the labels we have

$$J(U, V) = \frac{1}{|\mathcal{D}|} \sum_{i,j,r \in \mathcal{D}} \left( r - \mathbf{u}^{(i)T} \mathbf{v}^{(j)} \right)^2 \quad (\text{V.1})$$

where  $\mathbf{u}^{(i)}$  is the transpose of the  $i$ -th row of  $U$ ,  $\mathbf{v}^{(j)}$  is the transpose of the  $j$ -th row of  $V$ .

SGD objective function: Just one tuple in  $\mathcal{D} : (r_{ij}, i, j)$

$$J(\mathbf{u}^{(i)}, \mathbf{v}^{(j)}) = \left( r_{ij} - \mathbf{u}^{(i)T} \mathbf{v}^{(j)} \right)^2 \quad (\text{V.2})$$

SGD gradient steps

$$\begin{aligned} \frac{\partial J}{\partial \mathbf{u}^{(j)}} &= - \left( r_{ij} - \mathbf{u}^{(i)T} \mathbf{v}^{(j)} \right) \mathbf{v}^{(j)} \\ \frac{\partial J}{\partial \mathbf{v}^{(j)}} &= - \left( r_{ij} - \mathbf{u}^{(i)T} \mathbf{v}^{(j)} \right) \mathbf{u}^{(i)} \end{aligned} \quad (\text{V.3})$$

## B. Ensemble Methods: Boosting & Bagging

Suppose we have a pool of  $T$  binary classifiers  $\mathcal{A} = \{h_1, \dots, h_T\}$ , where

$$h_t : \mathbb{R}^M \rightarrow \{+1, -1\}.$$

Let  $\beta \in (0, 1)$  be a hyperparameter, and let  $\alpha_t$  be the weight for classifier  $h_t$ .

---

### Algorithm 18 Weighted Majority Algorithm

---

- 1: **procedure** WEIGHTEDMAJORITY( $\mathcal{A}, \beta$ )
  - 2:   Initialize classifier weights  $\alpha_t \leftarrow 1, \forall t \in \{1, \dots, T\}$
  - 3:   **for** each training example  $(x, y)$  **do**
  - 4:     Predict the majority-vote class (splitting ties randomly):  
 $\hat{h}(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right)$
  - 5:     **if**  $\hat{h}(x) \neq y$  **then**
  - 6:       **for** each classifier  $t \in \{1, \dots, T\}$  **do**
  - 7:         **if**  $h_t(x) \neq y$  **then**
  - 8:          $\alpha_t \leftarrow \beta \alpha_t$
- 

#### 1. AdaBoost

AdaBoost is a boosting method that combines a weak learner (which returns a hypothesis only slightly better than random guessing) into a strong learner (which returns a hypothesis of arbitrarily low error).

---

### Algorithm 19 AdaBoost Algorithm

---

- 1: Given training examples  $\{(x_1, y_1), \dots, (x_N, y_N)\}$  where  $x_i \in \mathbb{R}^M$  and  $y_i \in \{-1, +1\}$
  - 2: Initialize  $D_1(i) \leftarrow \frac{1}{N}$
  - 3: **for**  $t = 1, \dots, T$  **do**
  - 4:   Train a weak learner using distribution  $D_t$
  - 5:   Get weak hypothesis  $h_t : \mathbb{R}^M \rightarrow \{-1, +1\}$  with weighted error  
 $\epsilon_t = \mathbb{P}_{i \sim D_t}[h_t(x_i) \neq y_i] = \sum_{i=1}^N D_t(i) \mathbb{1}(h_t(x_i) \neq y_i)$
  - 6:   Choose  $\alpha_t \leftarrow \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$
  - 7:   **for**  $i = 1, \dots, N$  **do**
  - 8:     Update  

$$D_{t+1}(i) \leftarrow \frac{D_t(i)}{Z_t} \begin{cases} e^{-\alpha_t}, & \text{if } h_t(x_i) = y_i \\ e^{\alpha_t}, & \text{if } h_t(x_i) \neq y_i \end{cases} = \frac{D_t(i) \exp(-\alpha_t y_i h_t(x_i))}{Z_t}$$

where  $Z_t$  is chosen so that  $D_{t+1}$  is a distribution
  - 9: Output the final hypothesis:  
 $H(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right)$
-

The AdaBoost classifier weight is

$$\alpha_t = \frac{1}{2} \ln \left( \frac{1 - \epsilon_t}{\epsilon_t} \right). \quad (\text{V.4})$$

It decreases as the weak learner error  $\epsilon_t$  increases, and  $\alpha_t = 0$  at  $\epsilon_t = 0.5$ .

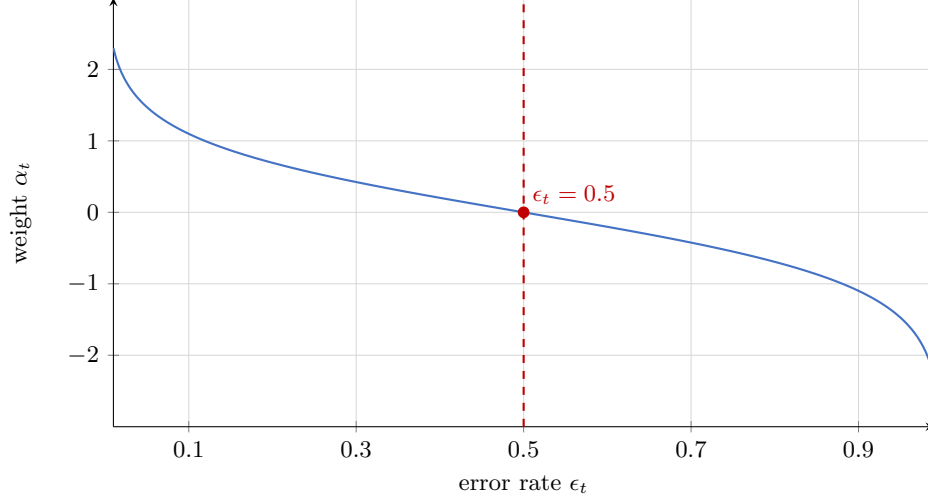


FIG. 19: AdaBoost weight  $\alpha_t$  as a function of weak learner error  $\epsilon_t$ . The dashed line marks  $\epsilon_t = 0.5$ , where  $\alpha_t = 0$ .

The AdaBoost distribution update is

$$\omega_i \leftarrow \omega_i \cdot \begin{cases} e^{-\Delta\alpha/Z} & , \text{ if } y^{(i)} = h_t(\mathbf{x}^{(i)}) \\ e^{\Delta\alpha/Z} & , \text{ if } y^{(i)} \neq h_t(\mathbf{x}^{(i)}) \end{cases} \quad (\text{V.5})$$

### C. Bagging

How can I obtain many classifiers/regressors to ensemble together?

We'll consider three possible answers:

1. (sample) bagging;
2. feature bagging (aka. random subspace method);
3. random forests (which combine sample bagging and feature bagging to train a "forest" of decision trees)

---

#### Algorithm 20 Sample Bagging

---

```

1: procedure SAMPLEBAGGING($\mathcal{D}, T, S$)
2: for $t = 1, \dots, T$ do
3: for $s = 1, \dots, S$ do
4: $i_s \sim \text{Uniform}(1, \dots, N)$
5: $\mathcal{S}_t = \{(x^{(i_s)}, y^{(i_s)})\}_{s=1}^S$
6: $h_t \leftarrow \text{train}(\mathcal{S}_t)$
7: return $\hat{h}(x) = \text{aggregate}(h_1, \dots, h_T)$

```

---

**Algorithm 21** Feature Bagging

---

```

1: procedure FEATUREBAGGING($\mathcal{D}, T, S$)
2: for $t = 1, \dots, T$ do
3: for $s = 1, \dots, S$ do
4: $m_s \sim \text{Uniform}(1, \dots, M)$
5: for $i = 1, \dots, N$ do
6: $\tilde{x}^{(i)} = [x_{m_1}^{(i)}, x_{m_2}^{(i)}, \dots, x_{m_S}^{(i)}]^T$
7: $\mathcal{D}_t = \{(\tilde{x}^{(i)}, y^{(i)})\}_{i=1}^N$
8: $h_t \leftarrow \text{train}(\mathcal{D}_t)$
9: return $\hat{h}(x) = \text{aggregate}(h_1, \dots, h_T)$

```

---

**random forests:** which combine sample bagging and feature bagging to train a forest of decision trees. draw a sample with replacement from the training examples,

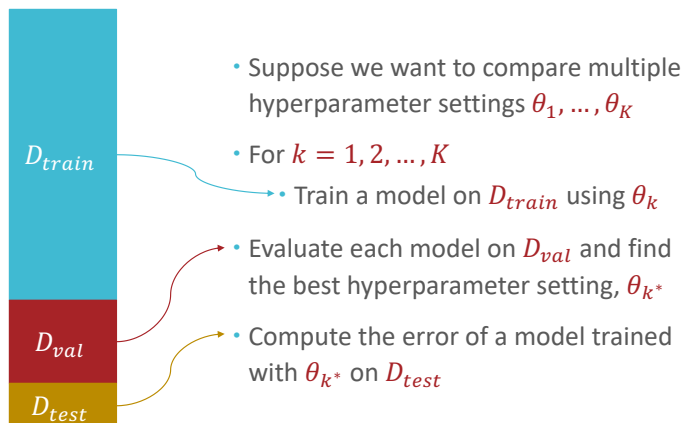
- recursively learn the decision tree
- but at each node when choosing a feature on which to split, first randomly sample a subset of the features, then pick the best feature from among that subset.

If  $B$  independent random variables  $x^{(1)}, x^{(2)}, \dots, x^{(B)}$  all have variance  $\sigma^2$ , then the variance of  $\frac{1}{B} \sum_{b=1}^B x^{(b)}$  is  $\frac{\sigma^2}{B}$

$$\begin{aligned} \text{Random forests} &= \text{sample bagging} + \text{feature bagging} \\ &= \text{bootstrap aggregating} + \text{split-feature randomization} \end{aligned} \quad (\text{V.6})$$

- Input:  $\mathcal{D} = \{(\mathbf{x}^{(n)}, \mathbf{y}^{(n)})\}_{n=1}^N, B, \rho$
- For  $b = 1, 2, \dots, B$ : Create a dataset,  $\mathcal{D}_b$ , by sampling  $N$  points from the original training data  $\mathcal{D}$  with replacement
- Learn a decision tree,  $t_b$ , using  $\mathcal{D}_b$  and the ID3 algorithm with split-feature randomization, sampling  $\rho$  features for each split
- Output:  $\bar{t} = f(t_1, \dots, t_B)$ , the aggregated hypothesis

Recall:  
Validation Sets



Out-of-bag(OOB) error, coming from computing an aggregated prediction for each  $\mathbf{x}^{(n)}$  using the trees in  $t^{(-n)}, \bar{t}^{(-n)}(\mathbf{x}^{(n)})$

$$E_{\text{OOB}} = \frac{1}{N} \sum_{n=1}^N \mathbb{1} \left( \bar{t}^{(-n)}(\mathbf{x}^{(n)}) \neq y^{(n)} \right)$$

Two main types of ensemble methods:

- Bagging: learns models in parallel by taking many subsets of the training data (useful for high variance);
- Boosting: learns models serially by reweighting the training data (useful for high bias).

### Bias-Variance Tradeoff

Suppose we have a regression dataset  $\mathcal{D} = \{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^N$  for which  $y^{(i)} = c^*(\mathbf{x}^{(i)}) + \epsilon$  where  $\epsilon \sim \text{Gaussian}(0, \sigma^2)$ .

We can decompose the mean squared error of a classifier  $h_{\theta}(\cdot)$  as follows:

$$\mathbb{E} \left[ (y - h_{\theta}(\mathbf{x}))^2 \right] = \text{Bias}^2 + \text{Variance} + \sigma^2 \quad (\text{V.7})$$

where

$$\begin{aligned} \text{Bias} &= \mathbb{E} [h_{\theta}(\mathbf{x})] - \mathbb{E} [c^*(\mathbf{x})] \\ \text{Variance} &= \mathbb{E} \left[ (\mathbb{E} [h_{\theta}(\mathbf{x})] - h_{\theta}(\mathbf{x}))^2 \right] \end{aligned} \quad (\text{V.8})$$

Above, all the expectations are under the uniform distribution over the dataset  $(\mathbf{x}, y) \sim \mathcal{D}$ .

## D. K-Means

---

### Algorithm 22 K-Means

---

```

1: procedure KMEANS($\{x^{(i)}\}_{i=1}^N, K$)
2: Initialize the K cluster centers $\{\mu_k\}_{k=1}^K$ (randomly if necessary)
3: repeat
4: for $i = 1, \dots, N$ do
5: Assign point $x^{(i)}$ to the nearest cluster center:
 $c^{(i)} \leftarrow \arg \min_{k \in \{1, \dots, K\}} \|x^{(i)} - \mu_k\|_2^2$
6: for $k = 1, \dots, K$ do
7: Re-estimate the k th cluster center:
 $\mu_k \leftarrow \frac{1}{|C_k|} \sum_{i: c^{(i)}=k} x^{(i)}$
8: until no object changed cluster membership in the last iteration
9: return $\{\mu_k\}_{k=1}^K$ and $\{c^{(i)}\}_{i=1}^N$

```

---

Alternating minimization

- a)  $\mathbf{z} = \underset{\mathbf{z}}{\operatorname{argmin}} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_{\mathbf{z}^{(i)}}\|_2^2$
- b)  $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K = \underset{\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K}{\operatorname{argmin}} \sum_{i=1}^N \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_{\mathbf{z}^{(i)}}\|_2^2$

**Algorithm 1.** Random Initialization: Select each cluster center uniformly at random from the data points in the training data;

**Algorithm 2.** Furthest Point Heuristic: 1) Pick the first cluster center  $\mathbf{c}_1$  randomly; 2) Pick each subsequent center  $\mathbf{c}_j$  so that it is as far as possible from the previously chosen centers  $\mathbf{c}_1, \mathbf{c}_2, \dots, \mathbf{c}_{j-1}$ ;

**Algorithm 3.** K-Means++: Let  $D(\mathbf{x})$  be the distance between a point  $x$  and its nearest center. Chose the next center proportional to  $D^2(\mathbf{x})$ .

Choose  $\mathbf{c}_1$  at random. For  $j = 2, \dots, K$  - Pick  $\mathbf{c}_j$  among  $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(n)}$  according to the distribution

$$\mathbf{P}(\mathbf{c}_j = \mathbf{x}^{(i)}) \propto \min_{j' < j} \|\mathbf{x}^{(i)} - \mathbf{c}_{j'}\|^2 = D^2(\mathbf{x}^i)$$

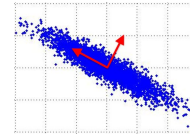
## E. Principal Component Analysis (PCA)

### Principal Component Analysis (PCA)

$(X^T X)v = \lambda v$ , so  $v$  (the first PC) is the eigenvector of sample covariance matrix  $X^T X$

Sample variance of projection  $v^T X^T X v = \lambda v^T v = \lambda$

Thus, the eigenvalue  $\lambda$  denotes the amount of variability captured along that dimension (aka amount of energy along that dimension).



Eigenvalues  $\lambda_1 \geq \lambda_2 \geq \lambda_3 \geq \dots$

- The 1<sup>st</sup> PC  $v_1$  is the eigenvector of the sample covariance matrix  $X^T X$  associated with the largest eigenvalue
- The 2<sup>nd</sup> PC  $v_2$  is the eigenvector of the sample covariance matrix  $X^T X$  associated with the second largest eigenvalue
- And so on ...

Slide from Nina Balcan

- PCA is an **unsupervised** method that finds orthogonal directions with the largest variance.
- First center the data:  $\tilde{x}^{(i)} = x^{(i)} - \mu$ , where  $\mu = \frac{1}{N} \sum_{i=1}^N x^{(i)}$ .
- Then compute the covariance matrix:

$$\Sigma = \frac{1}{N} \sum_{i=1}^N \tilde{x}^{(i)} \tilde{x}^{(i)T}$$

- The principal components are the eigenvectors of  $\Sigma$ , ordered by eigenvalues  $\lambda_1 \geq \lambda_2 \geq \dots$ .
- Keep the top  $k$  directions in  $U_k = [u_1, \dots, u_k]$ .
- Project to lower dimension:

$$z^{(i)} = U_k^T (x^{(i)} - \mu)$$

- Reconstruct by:

$$\hat{x}^{(i)} = \mu + U_k z^{(i)}$$

- PCA also gives the best rank- $k$  approximation in squared reconstruction error.
- Explained variance ratio:  $\lambda_j / \sum_{\ell} \lambda_{\ell}$ .
- Standardize features if they are on very different scales.
- Exam caveats: eigenvector signs are arbitrary, and PCA is sensitive to outliers.